

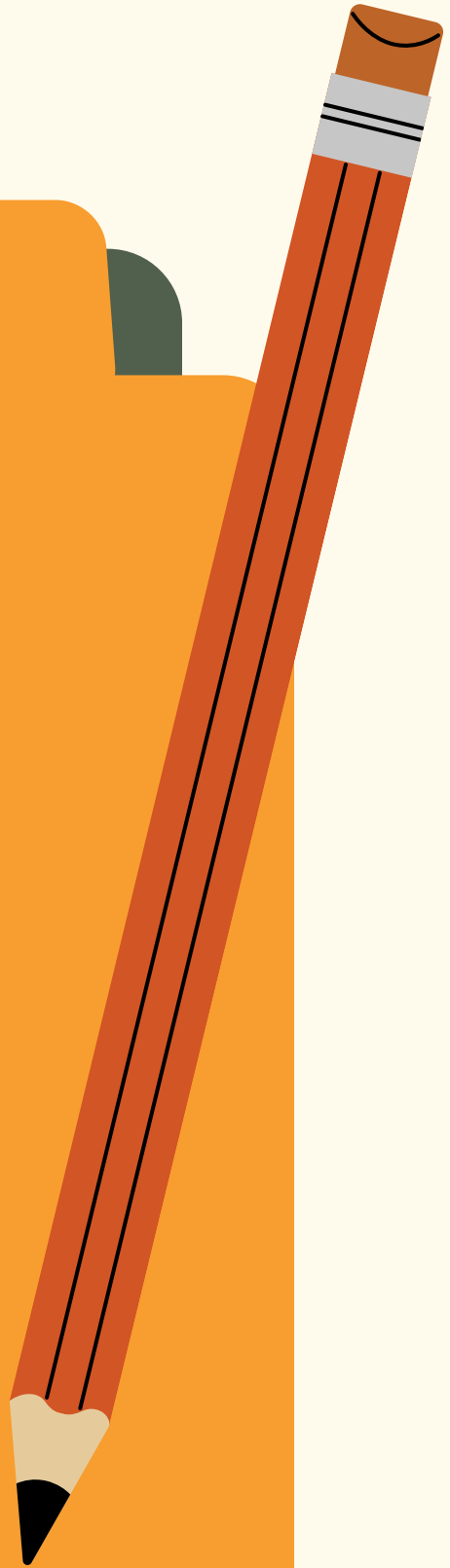
From numbers to AI

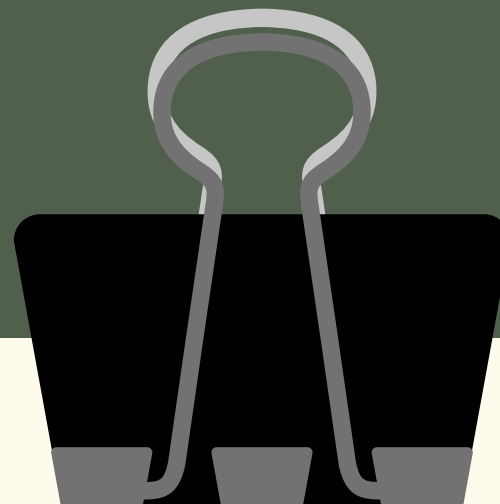
---

# PROGRAMMING

---

파이썬





# 질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



# Algorithm

이창민



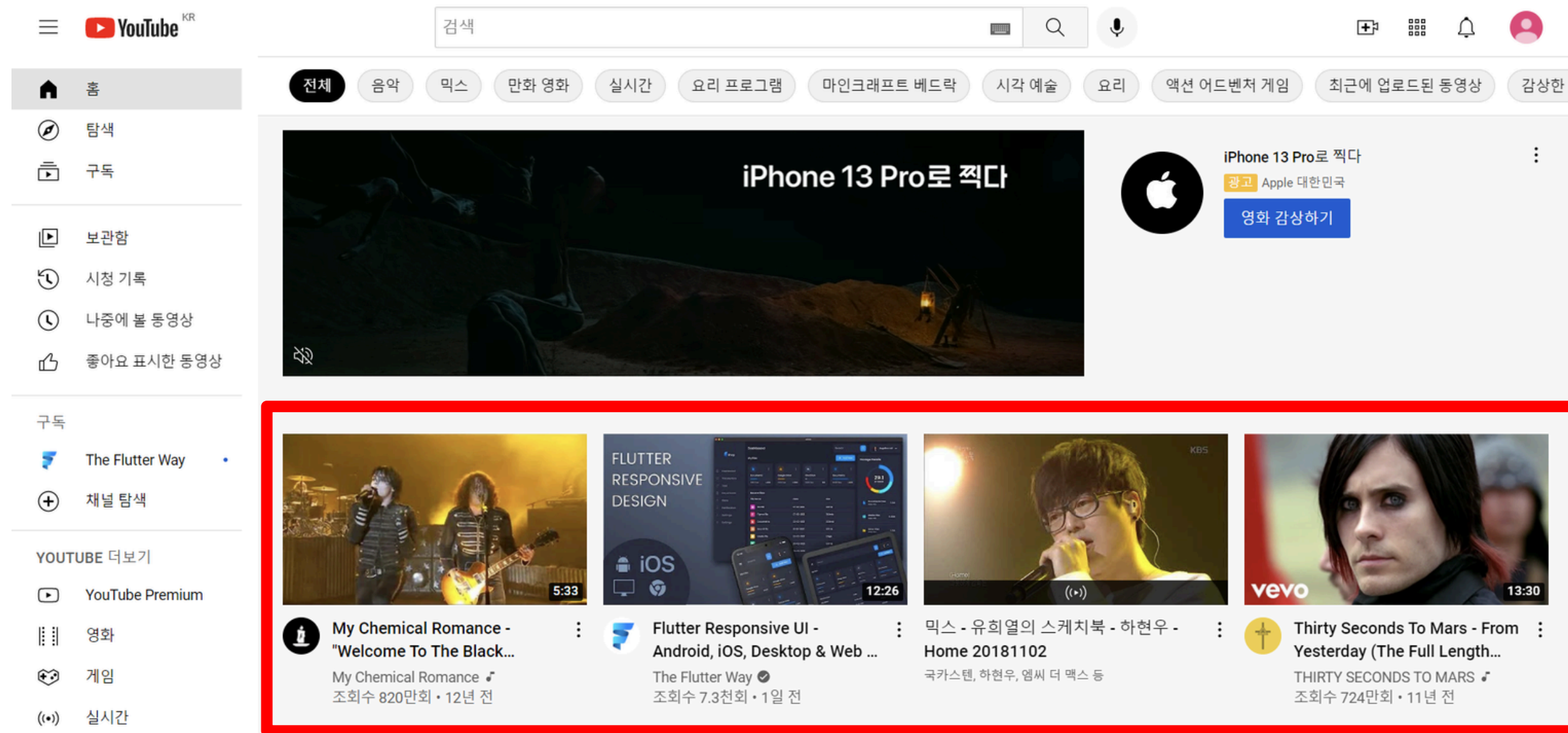
# 목차

- 알고리즘이란
- 효율적인 알고리즘이란
- Big-O Notation
- BFS와 DFS
- 최단 경로 탐색 알고리즘 -  $A^*$

# 알고리즘이란



# What is Algorithm?



“알 수 없는  
YouTube  
알고리즘이  
날 여기로  
이끌었다.”

# Algorithm

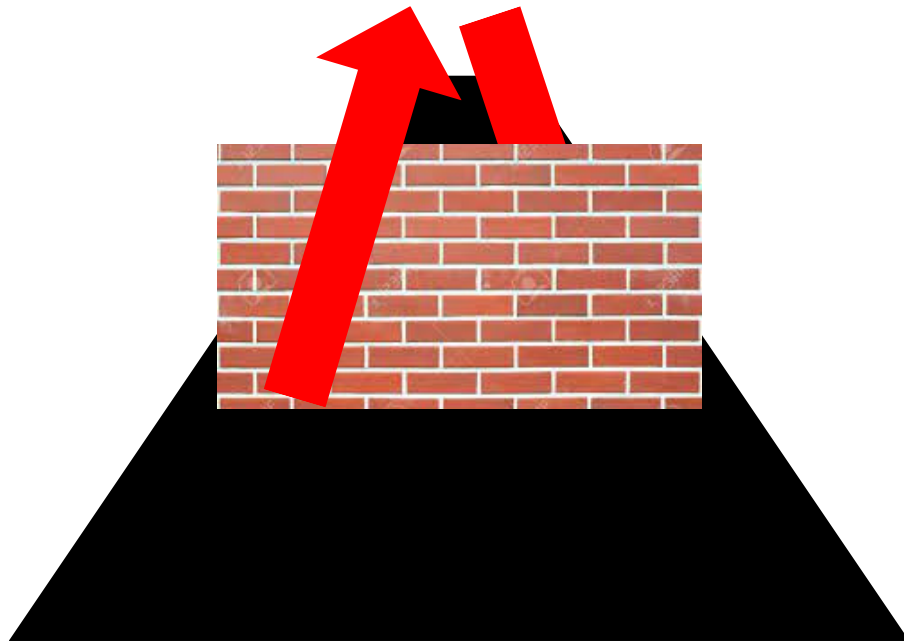
어떤 일을 수행하는 “방법”

# 알콰리즈미 아라비아 수학의 영웅

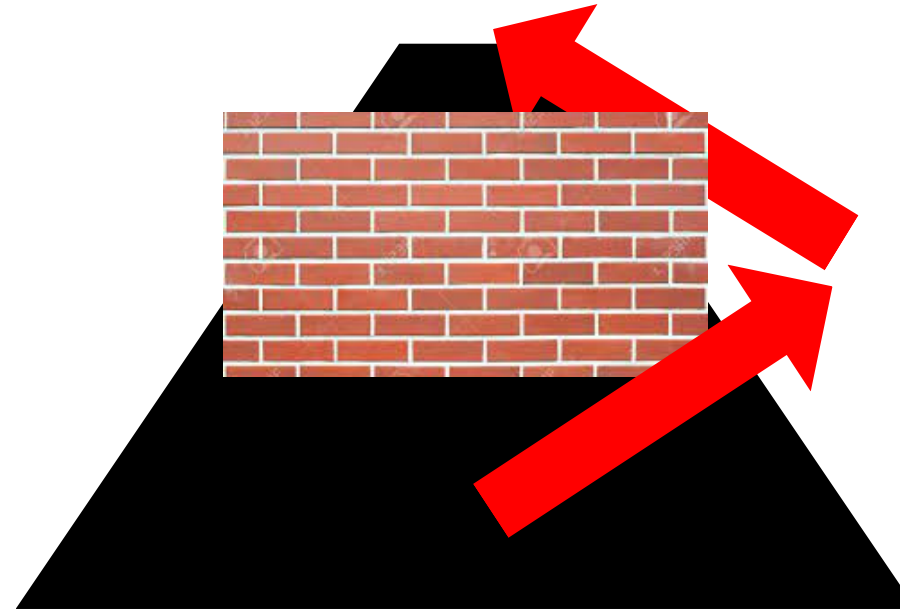


# What is Algorithm?

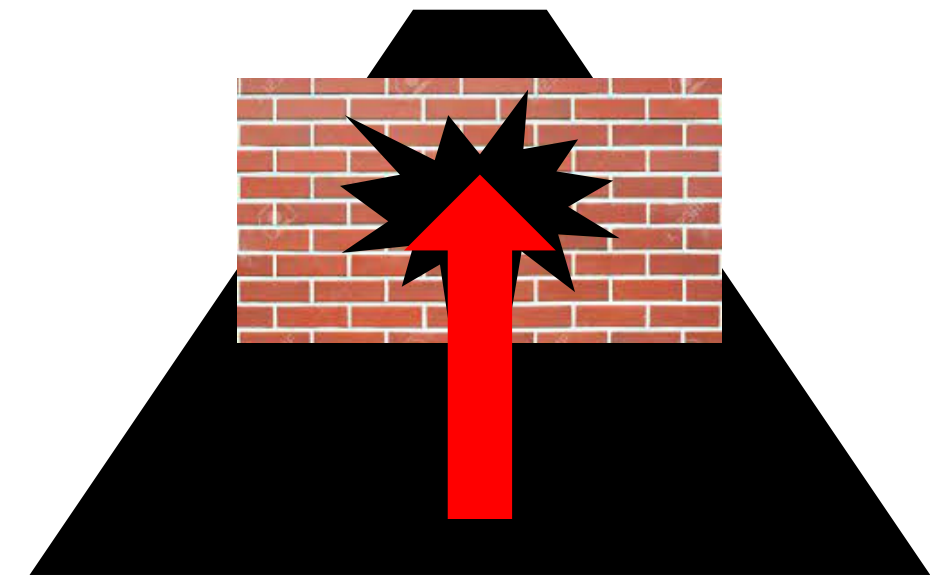
문제상황 : 길 한가운데에 벽이 있다. 어떻게 지나갈 것인가?



넘어가던가...



돌아가던가...



뚫고가던가...

뭐가 제일 “좋은” 방법일까?

# “Good” Algorithm?

문제상황 : 1부터 100까지 print 하시오.

```
1  print("1")
2  print("2")
3  print("3")
4  print("4")
5  print("5")
6  print("6")
7  :
8  print("95")
9  print("96")
10 print("97")
11 print("98")
12 print("99")
13 print("100")
```

```
1  for i in range(1,101):
2      print(i)
```

“단순작업의 반복보다는  
간결한 코드가 더 낫다!”

# “Good” Algorithm?

문제상황 : 1부터 n까지 합을 구하는 함수를 만드시오.

```
1 def sumOfN(n):  
2     theSum = 0  
3     for i in range(1,n+1):  
4         theSum = theSum + i  
5  
6     return theSum  
7  
8 print(sumOfN(10))
```

```
1 def foo(tom):  
2     fred = 0  
3     for bill in range(1,tom+1):  
4         barney = bill  
5         fred = fred + barney  
6  
7     return fred  
8  
9 print(foo(10))
```

“뭐 하는 코드인지 한 눈에 보이는 코드가 더 낫다!”

# “Good” Algorithm?

문제상황 : 1부터 n까지 모든 소수를 출력하는 함수를 만드시오.

```
1 def prime_printer(n):
2     for i in range(1,n+1):
3         detector = 0
4         for k in range(2,i):
5             if i%k == 0:
6                 detector += 1
7         if detector == 0:
8             print(i)
```

```
1 def prime_printer(n):
2     prime_list = []
3     for i in range(2,n+1):
4         detector = 0
5         for k in prime_list:
6             if i%k == 0:
7                 detector += 1
8         if detector == 0:
9             prime_list.append(i)
10            print(i)
```

“어느 쪽이 더 나은 코드일까?”

# 좋은 알고리즘이란?

---

- 간결성
- 가시성 (뭘 하는 코드인가?)
- 걸리는 시간이 짧은가?
- 메모리 이용이 효율적인가?



보통 판단하는 “기준”들

→ 무엇이 주가 될지는 **상황마다 다르다!**

# 문제를 풀어보자

자료형과 제어문, 함수를 이용하여 문제를 풀어보자

<https://blog.naver.com/nkj2001/222698193629>

|                                                                                                     |                                                                                                                                  |                                                              |
|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| *<br>***<br>*****<br>*****<br>*          *<br>***         ***<br>*****     *****<br>*****     ***** | 공백:8 별:1<br>공백:7 별:3<br>공백:6 별:5<br>공백:5 별:7<br>공백:4 별:1 공백:7 별:1<br>공백:3 별:3 공백:5 별:3<br>공백:2 별:5 공백:3 별:5<br>공백:1 별:7 공백:1 별:7 | i==0<br>i==1<br>i==2<br>i==3<br>i==4<br>i==5<br>i==6<br>i==7 |
|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|

```
a = 8
for i in range(a):
    if i < (a//2):
        for x in range(a-i):
            print(" ", end="")
        for x in range(i*2+1):
            print("*", end="")
    else:
        for x in range(a - i):
            print(" ", end="")
        for x in range(i*2+1 - a):
            print("*", end="")
        for x in range(a - (i*2+1 - a)):
            print(" ", end="")
        for x in range(i*2+1 - a):
            print("*", end="")
    print()
```

## 짝수인지 홀수인지 판별하기

- 문제 설명: 사용자로부터 정수를 하나 입력받아, 그 수가 짝수인지 홀수인지 출력하는 함수를 작성하시오.
  - 짝수면 "Even", 홀수면 "Odd" 를 출력해야 한다.

```
def check_even_odd():  
    num = int(input("숫자를 입력하세요: "))  
    if num % 2 == 0:  
        print("Even")  
    else:  
        print("Odd")  
  
check_even_odd()
```

## 짝수의 합 구하기

- 문제 설명: 사용자로부터 정수 N을 입력받아, 1부터 N까지의 숫자 중 짝수들의 합을 구하여 출력하는 함수를 작성하시오.

```
def sum_of_evens():  
    N = int(input("정수를 입력하세요: "))  
    total = 0  
    for i in range(1, N + 1):  
        if i % 2 == 0:  
            total += i  
    print("짝수의 합:", total)  
  
sum_of_evens()
```

## 양수만 골라내기

- 문제 설명: 사용자로부터 정수들을 공백으로 구분하여 한 줄로 입력받은 후, 그 중 양수만 리스트로 추출하여 출력하는 함수를 작성하시오.

```
def filter_positives():  
    nums = input("정수들을 공백으로 입력하세요: ")  
    num_list = list(map(int, nums.split()))  
    positive_list = []  
  
    for num in num_list:  
        if num > 0:  
            positive_list.append(num)  
  
    print("양수 목록:", positive_list)  
  
filter_positives()
```

## 홀수, 짝수, 양수, 음수 따로 분리하기

- 문제 설명: 사용자로부터 정수들을 공백으로 입력 받아, 짝수 리스트, 홀수 리스트, 양수 리스트, 음수 리스트를 각각 추출하여 출력하는 프로그램을 작성 하시오. 각 기능을 별도의 함수로 작성하고, 메인 함수에서 이들을 호출해 결과를 출력하도록 하시오.

```
def get_even(nums):  
    return [n for n in nums if n % 2 == 0]  
  
def get_odd(nums):  
    return [n for n in nums if n % 2 != 0]  
  
def get_positive(nums):  
    return [n for n in nums if n > 0]  
  
def get_negative(nums):  
    return [n for n in nums if n < 0]  
  
def main():  
    user_input = input("정수들을 공백으로 입력하세요: ")  
    num_list = list(map(int, user_input.split()))  
  
    print("짝수:", get_even(num_list))  
    print("홀수:", get_odd(num_list))  
    print("양수:", get_positive(num_list))  
    print("음수:", get_negative(num_list))  
  
main()
```

## 단어 개수 세기

- 문제 설명: 사용자로부터 문장을 입력받아, 각 단어가 몇 번 나왔는지 세어서 딕셔너리로 출력하는 함수를 작성하시오. (대소문자는 구분하지 않고, 문장 부호는 없다고 가정)

```
def count_words():  
    sentence = input("문장을 입력하세요: ").lower()  
    words = sentence.split()  
  
    word_count = {}  
    for word in words:  
        if word in word_count:  
            word_count[word] += 1  
        else:  
            word_count[word] = 1  
  
    print("단어 빈도수:", word_count)  
  
count_words()
```

## 학생 점수 등급 나누기

- 문제 설명: 학생들의 이름과 점수를 입력받고, 점수에 따라 다음 기준으로 등급(A/B/C/F)을 매겨 딕셔너리 형태로 출력하는 프로그램을 작성하시오.
  - 90점 이상: A
  - 80점 이상: B
  - 70점 이상: C
  - 그 외: F
- 요구 사항:
  - 이름과 점수를 '이름 점수' 형태로 공백 기준 여러 개 입력받음
  - 예: Tom 95 Jerry 82 Anna 67
  - 결과는 {이름: 등급} 형태로 출력

```
def grade_students():  
    data = input("학생 이름과 점수를 입력하세요: ") # 예: Tom 95 Jerry 82 Anna 67  
    parts = data.split()  
  
    grade_dict = {}  
    for i in range(0, len(parts), 2):  
        name = parts[i]  
        score = int(parts[i + 1])  
  
        if score >= 90:  
            grade = 'A'  
        elif score >= 80:  
            grade = 'B'  
        elif score >= 70:  
            grade = 'C'  
        else:  
            grade = 'F'  
  
        grade_dict[name] = grade  
  
    print("학생 등급:", grade_dict)  
  
grade_students()
```

## 중복 제거 후 정렬하기

- 문제 설명: 사용자로부터 정수를 공백으로 여러 개 입력받고, 중복을 제거한 후 오름차순 정렬된 리스트를 출력하는 함수를 작성하시오.

```
def dedup_and_sort():  
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))  
    unique_sorted = sorted(set(nums))  
    print("중복 제거 및 정렬 결과:", unique_sorted)  
  
dedup_and_sort()
```

## 짝수 리스트와 홀수 리스트 나누기

- 문제 설명: 사용자로부터 정수들을 공백으로 입력받아, 짝수 리스트와 홀수 리스트로 나누고 각각 출력하는 함수를 작성하시오.

```
def split_even_odd():  
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))  
    even_list = []  
    odd_list = []  
  
    for num in nums:  
        if num % 2 == 0:  
            even_list.append(num)  
        else:  
            odd_list.append(num)  
  
    print("짝수 리스트:", even_list)  
    print("홀수 리스트:", odd_list)  
  
split_even_odd()
```

## 최댓값과 최솟값 찾기

- 문제 설명: 사용자로부터 정수들을 공백으로 입력받아, 리스트의 최댓값과 최솟값을 찾아 출력하는 함수를 작성하시오. 함수는 두 값을 튜플로 리턴해야 함.

```
def find_max_min(nums):  
    max_val = max(nums)  
    min_val = min(nums)  
    return (max_val, min_val)  
  
def main():  
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))  
    result = find_max_min(nums)  
    print("최댓값:", result[0])  
    print("최솟값:", result[1])  
  
main()
```

## 숫자 등장 횟수 세기

- 문제 설명: 사용자로부터 정수들을 공백으로 입력받고, 각 숫자가 몇 번씩 나왔는지 딕셔너리로 출력하는 함수를 작성하시오.

```
def count_occurrences():  
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))  
    count_dict = {}  
  
    for num in nums:  
        if num in count_dict:  
            count_dict[num] += 1  
        else:  
            count_dict[num] = 1  
  
    print("숫자별 등장 횟수:", count_dict)  
  
count_occurrences()
```

## 모음 개수 세기

- 문제 설명: 사용자로부터 문자열을 입력받아, 그 안에 들어있는 모음(a, e, i, o, u)의 개수를 세어 출력하는 함수를 작성하시오. (대소문자 구분 없이 모두 카운트)

```
def count_vowels():  
    text = input("문자열을 입력하세요: ").lower()  
    vowels = 'aeiou'  
    count = 0  
  
    for char in text:  
        if char in vowels:  
            count += 1  
  
    print("모음 개수:", count)  
  
count_vowels()
```

## 문자열 뒤집기

- 문제 설명: 사용자로부터 문자열을 입력받아, 그 문자열을 뒤집어서 출력하는 함수를 작성하시오.

```
def reverse_string():  
    text = input("문자열을 입력하세요: ")  
    reversed_text = text[::-1]  
    print("뒤집은 문자열:", reversed_text)  
  
reverse_string()
```

## 특정 단어 등장 횟수 세기

- 문제 설명: 사용자로부터 문장과 검색할 단어를 각각 입력받아, 해당 단어가 문장에 몇 번 등장하는지 출력하는 함수를 작성하시오. (대소문자 구분 없이, 단어 단위만 매칭함)

```
def count_specific_word():  
    sentence = input("문장을 입력하세요: ").lower()  
    target = input("찾을 단어를 입력하세요: ").lower()  
    words = sentence.split()  
  
    count = 0  
    for word in words:  
        if word == target:  
            count += 1  
  
    print(f"'{target}'의 등장 횟수:", count)  
  
count_specific_word()
```

## 리스트의 최빈값 구하기

- 문제 설명: 사용자로부터 정수들을 공백으로 입력받아, 가장 많이 등장한 숫자(최빈값)를 출력하는 함수를 작성하시오. (동점이 있어도 그 중 아무거나 출력하면 됨)

```
def find_mode():
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))
    freq = {}

    for num in nums:
        if num in freq:
            freq[num] += 1
        else:
            freq[num] = 1

    max_freq = max(freq.values())

    for key in freq:
        if freq[key] == max_freq:
            print("최빈값:", key)
            break # 여러 개면 그 중 하나만 출력

find_mode()
```

## 숫자 맞추기 게임

- 문제 설명: 컴퓨터가 1부터 100 사이의 정수 중 하나를 랜덤하게 선택한다. 사용자는 숫자를 입력하며 추측하고, 컴퓨터는 입력한 숫자가 높은지/낮은지/정답인지 알려준다. 정답을 맞힐 때까지 계속 입력받는다.

```
import random

def number_guessing_game():
    answer = random.randint(1, 100)
    tries = 0

    while True:
        guess = int(input("숫자를 맞춰보세요 (1~100): "))
        tries += 1

        if guess < answer:
            print("너무 낮아요!")
        elif guess > answer:
            print("너무 높아요!")
        else:
            print(f"정답입니다! {tries}번 만에 맞췄어요 🎉")
            break

number_guessing_game()
```

## 최고 점수자 찾기

- 문제 설명: 사용자로부터 “이름 점수” 형식으로 여러 사람의 정보를 공백으로 입력받는다. 이 중에서 가장 점수가 높은 사람의 이름과 점수를 출력하시오.

```
def find_top_scorer():
    data = input("이름과 점수를 입력하세요 (예: Kim 85 Lee 92 Park 78): ").split()
    max_score = -1
    top_name = ""

    for i in range(0, len(data), 2):
        name = data[i]
        score = int(data[i + 1])

        if score > max_score:
            max_score = score
            top_name = name

    print(f"최고 점수자: {top_name} ({max_score}점)")

find_top_scorer()
```

## 가장 많이 나온 알파벳 찾기

- 문제 설명: 사용자로부터 문자열을 입력받아, 그 안에서 가장 많이 등장한 알파벳 한 글자를 출력하시오.
- 대소문자는 구분하지 않음. 공백, 숫자, 기호 등은 무시함. 동점이면 그 중 아무거나 출력해도 됨

```
def most_frequent_letter():
    text = input("문자열을 입력하세요: ").lower()
    freq = {}

    for char in text:
        if char.isalpha():
            if char in freq:
                freq[char] += 1
            else:
                freq[char] = 1

    max_count = max(freq.values())

    for char in freq:
        if freq[char] == max_count:
            print("가장 많이 나온 알파벳:", char)
            break # 동점이면 아무거나

most_frequent_letter()
```

## 두 번째로 큰 수 찾기 (정렬 없이)

- 문제 설명: 사용자로부터 정수들을 공백으로 입력받고,
- 정렬을 사용하지 않고, 두 번째로 큰 수를 찾아 출력하시오.

```
def find_second_largest():  
    nums = list(map(int, input("정수들을 공백으로 입력하세요: ").split()))  
  
    first = second = float('-inf')  
  
    for num in nums:  
        if num > first:  
            second = first  
            first = num  
        elif first > num > second:  
            second = num  
  
    if second == float('-inf'):  
        print("두 번째로 큰 수가 없습니다.")  
    else:  
        print("두 번째로 큰 수:", second)  
  
find_second_largest()
```

## 가장 긴 문장 찾기

- 문제 설명: 사용자로부터 여러 문장을 . 기호로 구분하여 입력받고, 그 중 가장 긴 문장을 찾아 출력하시오.
- ※ 문장 끝의 마침표(.)는 제거해서 비교하고, 가장 긴 문장은 출력할 때 양쪽 공백을 제거한 형태로 출력하시오.

```
def find_longest_sentence():  
    line = input("문장들을 '.'로 구분하여 입력하세요: ")  
    sentences = line.split('.')  
  
    longest = ""  
    for s in sentences:  
        s = s.strip()  
        if len(s) > len(longest):  
            longest = s  
  
    print("가장 긴 문장:", longest)  
  
find_longest_sentence()
```

## 점수 리포트 생성기

- 문제 설명: 사용자로부터 "이름 점수" 형식의 데이터를 한 줄에 여러명 입력받고, 다음 정보를 출력하는 리포트 생성 프로그램을 작성하시오.
- 출력 항목:
  - 총 학생 수
  - 평균 점수
  - 최고 점수 학생
  - 60점 미만인 낙제자 목록

```
def generate_score_report():  
    data = input("학생들의 이름과 점수를 입력하세요 (예: Kim 85 Lee 92 Park 55): ").split()  
  
    students = []  
    for i in range(0, len(data), 2):  
        name = data[i]  
        score = int(data[i + 1])  
        students.append((name, score))  
  
    total_students = len(students)  
    total_score = sum(score for _, score in students)  
    avg_score = total_score / total_students  
  
    top_student = max(students, key=lambda x: x[1])  
    fail_list = [name for name, score in students if score < 60]  
  
    print("\n📊 점수 리포트")  
    print("총 학생 수:", total_students)  
    print("평균 점수: {:.2f}".format(avg_score))  
    print("최고 점수자: {} ({}점)".format(top_student[0], top_student[1]))  
    print("낙제자 목록:", fail_list if fail_list else "없음")  
  
generate_score_report()
```

## 회문(Palindrome) 판별기

- 문제 설명: 사용자로부터 문자열을 입력받아, 그 문자열이 앞에서 읽어도 뒤에서 읽어도 같은 회문 (palindrome) 인지 판별하는 프로그램을 작성하시오.
- 공백, 대소문자 구분 없이 비교하며, 결과는 True 또는 False로 출력한다.

```
def is_palindrome():  
    text = input("문자열을 입력하세요: ")  
    filtered = ''.join(c.lower() for c in text if c.isalnum())  
    result = filtered == filtered[::-1]  
    print("회문 여부:", result)  
  
is_palindrome()
```

## bin 없이 2진수로 변환하기 (음수 포함)

- 문제 설명: 사용자로부터 정수를 입력 받아, bin() 함수를 사용하지 않고, 해당 정수를 직접 2진수 문자열로 변환하시오.
  - 양수는 일반적인 이진수로
  - 0은 '0' 음수는 2의 보수가 아닌, 그냥 '-' 기호를 붙인 문자열 형태로 처리 (예: -5 → '-101')

```
def to_binary_string(n):  
    if n == 0:  
        return '0'  
  
    is_negative = n < 0  
    n = abs(n)  
    result = ''  
  
    while n > 0:  
        result = str(n % 2) + result  
        n //= 2  
  
    if is_negative:  
        result = '-' + result  
  
    return result  
  
def main():  
    num = int(input("정수를 입력하세요: "))  
    binary = to_binary_string(num)  
    print("이진수:", binary)  
  
main()
```

## 문자열 압축 (Run-Length Encoding)

- 문제 설명: 사용자로부터 문자열을 입력받아, 연속된 문자가 반복되는 횟수만큼 압축한 문자열을 출력하시오.
  - 같은 문자가 연속으로 여러 번 등장하면 문자+횟수로 표현
  - 한 번만 등장한 문자는 그대로 출력
  - 대소문자 구분함
- 예시:
  - 입력: "aaabbcaa"
  - 출력: "a3b2ca2"

```
def run_length_encoding(text):  
    if not text:  
        return ''  
  
    result = ''  
    prev_char = text[0]  
    count = 1  
  
    for i in range(1, len(text)):  
        if text[i] == prev_char:  
            count += 1  
        else:  
            result += prev_char + (str(count) if count > 1 else '')  
            prev_char = text[i]  
            count = 1  
  
    result += prev_char + (str(count) if count > 1 else '')  
    return result  
  
def main():  
    s = input("문자열을 입력하세요: ")  
    encoded = run_length_encoding(s)  
    print("압축 결과:", encoded)  
  
main()
```

## 괄호 짝 검사기 (Valid Parentheses)

- 문제 설명: 사용자로부터 괄호 문자열을 입력받아, 모든 괄호가 올바르게 열리고 닫혔는지를 검사하는 프로그램을 작성하시오.
  - 검사할 괄호: (), {}, []
  - 올바른 괄호라면 True, 아니면 False 출력
  - 스택을 사용해서 구현할 것
- 예시:
  - 입력: "({[]})" → 출력: True
  - 입력: "([)]" → 출력: False

```
def is_valid_brackets(s):  
    stack = []  
    pair = {')': '(', '}': '{', ']': '['}  
  
    for char in s:  
        if char in '({[':  
            stack.append(char)  
        elif char in ')}]':  
            if not stack or stack[-1] != pair[char]:  
                return False  
            stack.pop()  
    return not stack  
  
def main():  
    expr = input("괄호 문자열을 입력하세요: ")  
    print("올바른 괄호 구조:", is_valid_brackets(expr))  
  
main()
```

## 로또 번호 생성기 (중복 없이 정렬 포함)

- 문제 설명: 로또 번호를 무작위로 6개 생성하는 함수를 작성하시오.
  - 각 번호는 1~45 사이의 정수
  - 중복 없이 6개
  - 오름차순 정렬된 리스트를 출력할 것

```
import random

def generate_lotto():
    numbers = set()
    while len(numbers) < 6:
        num = random.randint(1, 45)
        numbers.add(num)

    lotto_numbers = sorted(numbers)
    print("🎯 로또 번호:", lotto_numbers)

generate_lotto()
```

## 애너그램 판별기

- 문제 설명: 두 개의 단어를 입력받아, 두 단어가 애너그램(anagram) 관계인지 확인하는 함수를 작성하시오.
  - 애너그램: 동일한 문자를 다른 순서로 나열한 단어
  - 대소문자 무시, 공백 제거
  - 결과는 True 또는 False 출력
- 예시:
  - 입력: "listen" vs "silent" → 출력: True
  - 입력: "hello" vs "world" → 출력: False

```
def is_anagram(word1, word2):  
    w1 = sorted(word1.replace(" ", "").lower())  
    w2 = sorted(word2.replace(" ", "").lower())  
    return w1 == w2  
  
def main():  
    word1 = input("첫 번째 단어를 입력하세요: ")  
    word2 = input("두 번째 단어를 입력하세요: ")  
    print("애너그램 여부:", is_anagram(word1, word2))  
  
main()
```

## 팩토리얼 계산기 (재귀 + 반복 + 예외처리)

- 문제 설명: 사용자로부터 정수를 입력 받아 팩토리얼(factorial) 값을 출력하시오.
- 다음 요구사항을 모두 만족해야 함:
  - 음수를 입력하면 "음수는 팩토리얼을 계산할 수 없습니다." 출력
  - 0 또는 양수는 두 가지 방법으로 팩토리얼 계산:
    - 재귀 함수
    - 반복문
  - 두 방법의 결과를 모두 출력할 것

```
def factorial_recursive(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial_recursive(n - 1)  
  
def factorial_iterative(n):  
    result = 1  
    for i in range(2, n + 1):  
        result *= i  
    return result  
  
def main():  
    try:  
        n = int(input("정수를 입력하세요: "))  
        if n < 0:  
            print("음수는 팩토리얼을 계산할 수 없습니다.")  
        else:  
            print("재귀 방식:", factorial_recursive(n))  
            print("반복 방식:", factorial_iterative(n))  
    except ValueError:  
        print("유효한 정수를 입력해주세요.")
```

main()

## 피보나치 수열 (메모이제이션 사용)

- 문제 설명: 사용자로부터 정수 N을 입력받아, 피보나치 수열의 N번째 항까지 출력하는 프로그램을 작성하시오.
- 단, 다음 조건을 만족해야 함:
  - $\text{fibonacci}(0) = 0$ ,  $\text{fibonacci}(1) = 1$ , 그 이후는  $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$
  - 단순 재귀 대신 메모이제이션(저장)을 이용해 중복 계산 방지
  - 결과는 0번째 ~ N번째 항까지 리스트로 출력

```
def fibo(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n == 0:  
        memo[0] = 0  
    elif n == 1:  
        memo[1] = 1  
    else:  
        memo[n] = fibo(n - 1, memo) + fibo(n - 2, memo)  
    return memo[n]  
  
def generate_fibonacci_sequence(n):  
    return [fibo(i) for i in range(n + 1)]  
  
def main():  
    try:  
        n = int(input("피보나치 몇 번째 항까지 출력할까요? "))  
        if n < 0:  
            print("음수는 허용되지 않습니다.")  
        else:  
            sequence = generate_fibonacci_sequence(n)  
            print("피보나치 수열:", sequence)  
    except ValueError:  
        print("유효한 정수를 입력해주세요.")  
  
main()
```

## 단어 등장 Top 3 통계

- 문제 설명: 사용자로부터 문장을 여러 줄 입력받아(빈 줄이면 종료), 모든 단어 중에서 가장 많이 등장한 단어 3개를 출력하시오.
  - 대소문자 구분 없이 비교
  - 마침표, 쉼표 등 문장부호 제거
  - 단어 개수는 내림차순 정렬 후 상위 3개 출력
  - 동점이면 아무거나 출력해도 됨

```
import string

def normalize(text):
    return text.translate(str.maketrans('', '', string.punctuation)).lower()

def get_top_words():
    print("문장을 입력하세요 (빈 줄 입력 시 종료):")
    word_count = {}

    while True:
        line = input()
        if line.strip() == "":
            break
        normalized = normalize(line)
        words = normalized.split()

        for word in words:
            word_count[word] = word_count.get(word, 0) + 1

    sorted_words = sorted(word_count.items(), key=lambda x: x[1], reverse=True)
    top3 = sorted_words[:3]

    print("\n🏆 가장 많이 등장한 단어 Top 3:")
    for word, count in top3:
        print(f"{word}: {count}회")

get_top_words()
```

## 키워드 기반 미니 챗봇

- 문제 설명: 사용자가 입력하는 문장에 따라 반응하는 간단한 키워드 기반 챗봇을 구현하시오.
- 요구사항:
  - 사용자의 입력을 분석하여 다음과 같이 반응함:
  - "안녕" → "안녕하세요! 😊"
  - "이름" → "저는 챗봇이에요!"
  - "날씨" → "날씨는 항상 마음속에 있어요 ☀️"
  - "종료" 또는 "bye" → "대화 종료합니다. 안녕히 가세요!" 출력 후 종료
  - 그 외 → "잘 이해하지 못했어요 😓"
  - 무한 루프로 동작하다가 종료 키워드가 들어오면 종료

이모티콘은 생략해도댐

```
def chatbot():  
    print("🤖 챗봇에게 말을 걸어보세요! (종료하려면 '종료' 또는 'bye')")  
  
    while True:  
        msg = input("👤 사용자: ").lower()  
  
        if "안녕" in msg:  
            print("🤖 챗봇: 안녕하세요! 😊")  
        elif "이름" in msg:  
            print("🤖 챗봇: 저는 챗봇이에요!")  
        elif "날씨" in msg:  
            print("🤖 챗봇: 날씨는 항상 마음속에 있어요 ☀️")  
        elif "종료" in msg or "bye" in msg:  
            print("🤖 챗봇: 대화 종료합니다. 안녕히 가세요!")  
            break  
        else:  
            print("🤖 챗봇: 잘 이해하지 못했어요 😓")  
  
chatbot()
```

## 텍스트 기반 미니 검색엔진

- 문제 설명: 여러 문장(또는 기사)을 미리 저장해놓고, 사용자가 단어를 입력하면 해당 단어가 포함된 문장들을 검색해서 출력하는 시스템을 만들자.
- 요구사항:
  - 문장들은 리스트로 미리 정의되어 있음
  - 검색어는 input으로 받음
  - 대소문자 구분 없이 단어가 단어 단위로 포함된 문장만 출력
  - 부분 문자열이 아닌 완전 일치 단어만 검색
  - 검색 결과가 없으면 "검색 결과가 없습니다." 출력
  - 검색은 반복적으로 가능하고, '종료' 입력 시 프로그램 종료

```
def normalize_sentence(sentence):
    import string
    return sentence.translate(str.maketrans('', '', string.punctuation)).lower()

def search_engine():
    documents = [
        "Python is a powerful programming language.",
        "Machine learning is a branch of artificial intelligence.",
        "Many developers love Python for its readability.",
        "Data science involves statistics and programming.",
        "Natural language processing helps computers understand human language."
    ]

    print("🔍 미니 검색엔진 (종료하려면 '종료' 입력)")

    while True:
        query = input("검색어를 입력하세요: ").lower().strip()
        if query == "종료":
            print("검색을 종료합니다.")
            break

        found = False
        for doc in documents:
            words = normalize_sentence(doc).split()
            if query in words:
                print("📄", doc)
                found = True

        if not found:
            print("검색 결과가 없습니다.")

search_engine()
```

## 텍스트 요약기 (단순형)

- 문제 설명: 사용자로부터 여러 문장을 입력받고, 중복된 문장은 제거하고 가장 많이 등장한 단어를 기준으로 해당 단어가 포함된 문장 중 앞에서부터 2개 문장을 출력하시오.
- 요구사항:
  - 문장은 . 으로 구분하여 한 줄로 입력받음
  - 마침표 제거 및 공백/대소문자 정리 필요
  - 동일 문장은 1번만 포함
  - 단어 빈도는 전체 문장에서 계산
  - 가장 많이 나온 단어가 포함된 문장 중 앞 2개 출력

```
import string
from collections import Counter

def summarize_text(text):
    # 마침표로 문장 나누기
    raw_sentences = [s.strip() for s in text.split('.') if s.strip()]
    # 중복 제거
    unique_sentences = list(dict.fromkeys(raw_sentences))

    # 전체 단어 수 세기
    all_words = []
    sentence_words = []

    for sentence in unique_sentences:
        cleaned = sentence.translate(str.maketrans('', '', string.punctuation)).lower()
        words = cleaned.split()
        sentence_words.append(words)
        all_words.extend(words)

    word_freq = Counter(all_words)
    most_common_word, _ = word_freq.most_common(1)[0]

    # 해당 단어 포함된 문장 중 앞에서 2개
    summary = []
    for i, words in enumerate(sentence_words):
        if most_common_word in words:
            summary.append(unique_sentences[i])
            if len(summary) == 2:
                break

    print("\n📝 요약 결과:")
    for line in summary:
        print("-", line)

def main():
    print("문장을 입력하세요 ('.'으로 구분, 한 줄):")
    text = input()
    summarize_text(text)

main()
```

## 텍스트 기반 퀴즈 시스템

- 문제 설명: 5개의 퀴즈가 순서대로 출제되며, 사용자 입력으로 정답을 받고 정답이면 "정답입니다!", 오답이면 "틀렸습니다. 정답은 \_\_입니다." 출력 마지막에 총 점수 출력 (1문제당 20점)

```
def quiz_game():
    questions = [
        {"question": "파이썬의 자료형 중 변경 불가능한 것은?", "answer": "tuple"},
        {"question": "HTML은 무엇의 약자인가?", "answer": "hypertext markup language"},
        {"question": "3의 제곱은?", "answer": "9"},
        {"question": "리스트에서 요소를 추가하는 함수는?", "answer": "append"},
        {"question": "파이썬에서 함수를 정의할 때 사용하는 키워드는?", "answer": "def"}
    ]

    print("🎓 퀴즈를 시작합니다!\n")
    score = 0

    for i, q in enumerate(questions):
        print(f"Q{i+1}. {q['question']}")
        user = input("👉 답변: ").strip().lower()

        if user == q["answer"]:
            print("✅ 정답입니다!\n")
            score += 1
        else:
            print(f"❌ 틀렸습니다. 정답은 '{q['answer']}'입니다.\n")

    print("🏁 퀴즈 종료!")
    print(f"당신의 총 점수는 {score * 20}점입니다. ({score}/5 정답)")

quiz_game()
```

## 일정 관리 콘솔 프로그램 (CRUD)

- 문제 설명: 사용자는 메뉴를 선택하여 일정 관리 기능을 사용할 수 있다.
- 기능: 일정 등록, 일정 조회, 일정 수정, 일정 삭제, 프로그램 종료
- 요구사항:
  - 일정은 날짜(str) → 내용(str) 형태로 저장 (예: '2024-04-13': 'AI 프로젝트 회의')
  - 모든 입력은 콘솔에서 수행
  - 없는 일정 수정/삭제 시 오류 메시지 출력
  - 최소한의 UI로 텍스트 기반 콘솔 시스템 구성

```
def print_menu():  
    print("\n📅 일정 관리 시스템")  
    print("1. 일정 등록")  
    print("2. 일정 조회")  
    print("3. 일정 수정")  
    print("4. 일정 삭제")  
    print("5. 종료")
```

```
def schedule_manager():  
    schedules = {}  
  
    while True:  
        print_menu()  
        choice = input("메뉴를 선택하세요 (1~5): ").strip()  
  
        if choice == '1':  
            date = input("날짜 입력 (예: 2024-04-13): ").strip()  
            content = input("일정 내용 입력: ").strip()  
            schedules[date] = content  
            print("✅ 일정이 등록되었습니다.")  
  
        elif choice == '2':  
            date = input("조회할 날짜 입력: ").strip()  
            if date in schedules:  
                print(f"📅 {date}: {schedules[date]}")  
            else:  
                print("❗ 해당 날짜의 일정이 없습니다.")  
  
        elif choice == '3':  
            date = input("수정할 날짜 입력: ").strip()  
            if date in schedules:  
                new_content = input("새 일정 내용 입력: ").strip()  
                schedules[date] = new_content  
                print("🔄 일정이 수정되었습니다.")  
            else:  
                print("❗ 수정할 일정이 존재하지 않습니다.")  
  
        elif choice == '4':  
            date = input("삭제할 날짜 입력: ").strip()  
            if date in schedules:  
                del schedules[date]  
                print("🗑️ 일정이 삭제되었습니다.")  
            else:  
                print("❗ 삭제할 일정이 존재하지 않습니다.")  
  
        elif choice == '5':  
            print("👋 일정 관리 프로그램을 종료합니다.")  
            break  
  
        else:  
            print("❗ 유효한 메뉴 번호를 입력하세요.")  
  
    schedule_manager()
```

# Big-O Notation

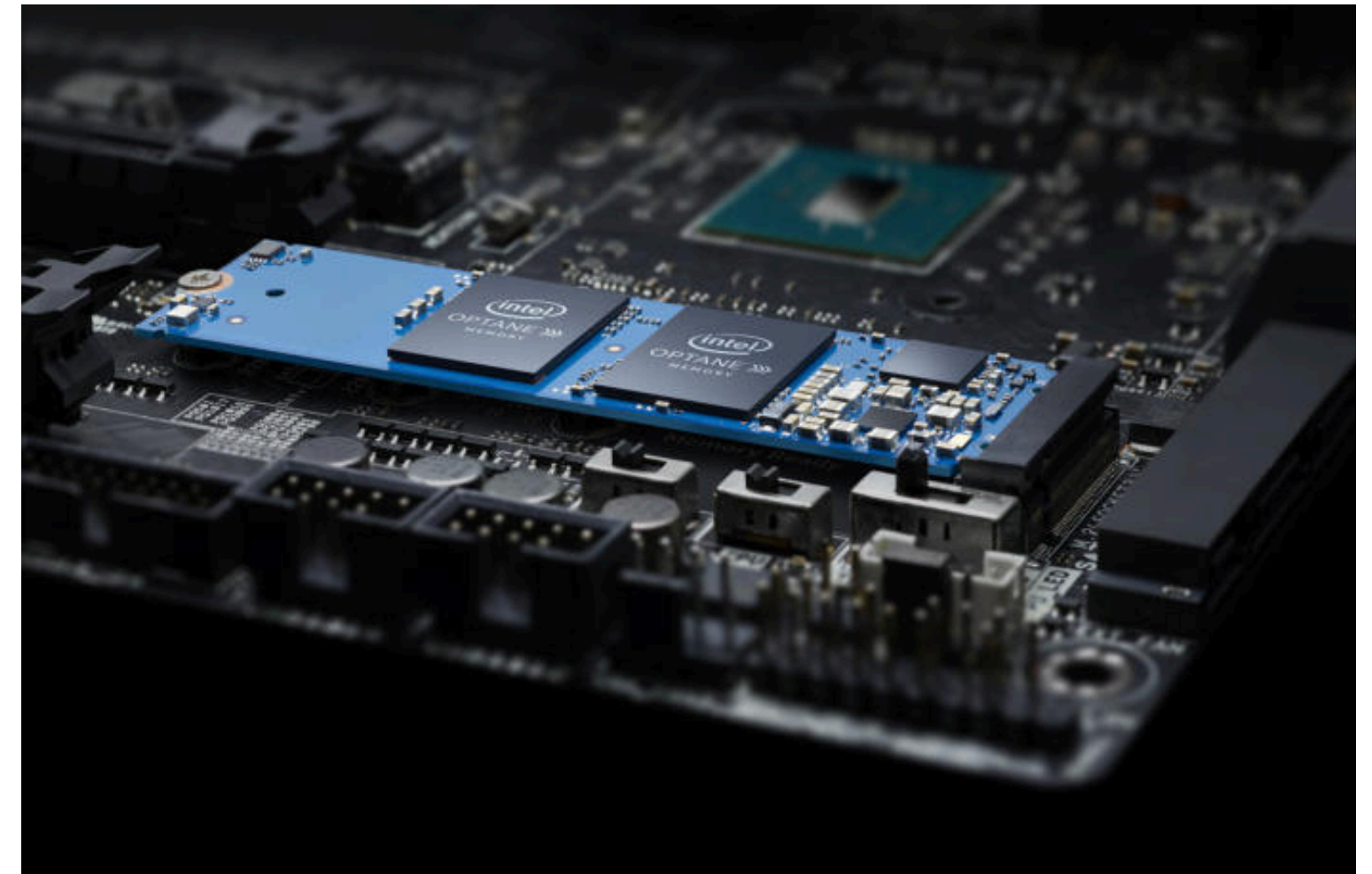
시간 복잡도

# Question!

매번 저렇게 시간을 구해서 비교해봐야만  
어느 쪽이 효율적인지 알 수 있나?

# 프로그램의 " 복잡도 "

- 시간 복잡도 (Time Complexity)
  - 해당 작업을 수행하기 위해 얼마만큼의 시간이 걸리는가?
  - **Mainly Discussed**
- 
- 공간 복잡도 (Space Complexity)
  - 해당 작업을 수행하기 위해 얼마만큼의 메모리를 사용하는가?
  - 기술의 발전으로 크게 고민 X  
(해야 하는 상황도 존재)



# 시간 복잡도 체크

```
1 import time
2
3 def prime_printer_1(n):
4     start = time.time()
5     prime_list = []
6     for i in range(2,n+1):
7         detector = 0
8         for k in prime_list:
9             if i%k == 0:
10                 detector += 1
11             if detector == 0:
12                 prime_list.append(i)
13     end = time.time()
14     return end-start
15
16 print("required %10.12f seconds" %prime_printer_1(100))
17 print("required %10.12f seconds" %prime_printer_1(1000))
18 print("required %10.12f seconds" %prime_printer_1(10000))
19 print("required %10.12f seconds" %prime_printer_1(100000))
```

Python - al.py:19 ✓

required 0.000000000000 seconds  
required 0.005992412567 seconds  
required 0.287708282471 seconds  
required 23.693246364594 seconds  
[Finished in 24.361s]

"23.69 sec"

```
1 import time
2
3 def prime_printer_2(n):
4     start = time.time()
5     prime_list = []
6     for i in range(1,n+1):
7         detector = 0
8         for k in range(2,i):
9             if i%k == 0:
10                 detector += 1
11             if detector == 0:
12                 prime_list.append(i)
13     end = time.time()
14     return end-start
15
16 print("required %10.12f seconds" %prime_printer_2(100))
17 print("required %10.12f seconds" %prime_printer_2(1000))
18 print("required %10.12f seconds" %prime_printer_2(10000))
19 print("required %10.12f seconds" %prime_printer_2(100000))
```

Python - al.py:19 ✓

required 0.000000000000 seconds  
required 0.034698247910 seconds  
required 2.420342445374 seconds  
required 312.855787277222 seconds  
[Finished in 315.724s]

"312.86 sec"

# Big-O Notation이란?

---

- 프로그램의 시간 복잡도를 나타내는데 쓰이는 기준
- 매번 걸린 시간을 출력하는 수고를 줄이기
- 한 눈에 복잡도를 계산할 수 있게 하는 방법

# Big-O Notation

- 코드의 “Step”의 수를 세서 표기

```
def prime_printer(n):  
    for i in range(1,n+1):  
        detector = 0  
        for k in range(2,i):  
            if i%k == 0:  
                detector += 1  
        if detector == 0:  
            print(i)
```

for문 안 Step 수  $\times n$

Step 수 +1

for문 안 Step 수  $\times (i-1)$

Step 수 +1 ( $i\%k == 0$ 인지 연산)

Step 수 +1

Step 수 +1 (detector == 0인지 확인)

Step 수 +1

# Big-O Notation

---

- 앞에서 계산한 의 최고 차항의 **차수**만 가져온다.
- : 시간 복잡도

```
1 def prime_printer(n):
2     for i in range(1,n+1):
3         detector = 0
4         for k in range(2,i):
5             if i%k == 0:
6                 detector += 1
7         if detector == 0:
8             print(i)
```

중첩된  
for문의 개수?!

# Question

- 왜 최고차항만 떼서 비교하는가? → 그래도  $n$ 이 커지면 유의미하기 때문!

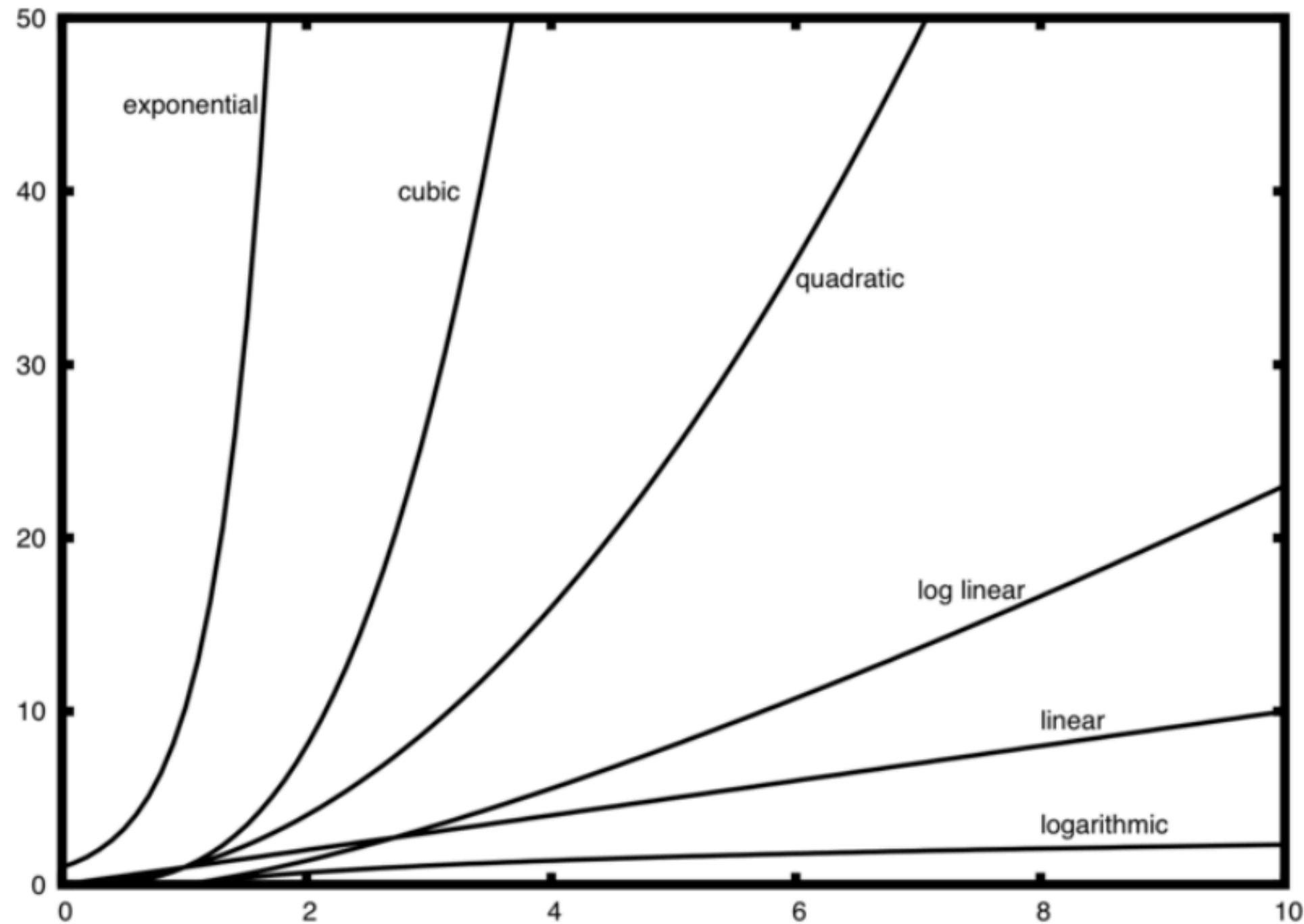


Figure 1: Plot of Common Big-O Functions

프로그램의 크기( $n$ )에 따라  
해당 프로그램이 실행되는데 걸  
리는 시간 그래프

# 다양한 Big-O functions

Table 1: Common Functions for Big-O

| f(n)       | Name        |
|------------|-------------|
| 1          | Constant    |
| $\log n$   | Logarithmic |
| $n$        | Linear      |
| $n \log n$ | Log Linear  |
| $n^2$      | Quadratic   |
| $n^3$      | Cubic       |
| $2^n$      | Exponential |

# 다양한 Big-O functions

- Constant Complexity –  $O(c)$  /  $O(1)$ 
  - Input의 수와 상관없이 알고리즘의 실행에 걸리는 스텝의 수가 일정하게 유지될 때

```
1 def constant_algo(input_):  
2     result = input_[0] * input_[0]  
3     return result  
4  
5 constant_algo([4, 5, 6, 8])
```

- 리스트 인덱싱의 시간복잡도는  $O(1)$
- 그 값을 찾아서 제공하는 스텝 하나, return하는 스텝 하나 총 2 스텝
- 리스트 길이가 매우 길어져도 인덱스로 값 하나 찾아서 제공해서 return할 뿐
- 해당 코드의 시간 복잡도는 상수(Constant Complexity) –  $O(c)$  /  $O(1)$

# 다양한 Big-O functions

- Linear Complexity –  $O(n)$

- 알고리즘의 실행에 필요한 스텝의 수가 인풋의 수와 비례하여 늘어나거나 줄어든 때

```
1 def linear_algo(input_):  
2     for item in input_:  
3         print(item)  
4  
5 linear_algo([4, 5, 6, 8])
```

- For문으로 input\_에 들어 있는 요소를 하나하나 출력
- Input 길이가 2면 두 번, 3이면 세 번, n이면 n번
- linear\_algo의 시간 복잡도는 linear(선형)이며, 이는  $O(n)$ 으로 나타낸다

# 다양한 Big-O functions

- Quadratic Complexity –  $O(n^2)$ 
  - 알고리즘의 실행에 필요한 스텝의 수가 인풋에 포함된 원소의 수의 2차 함수 형태일 때

```
1 def quadratic_algo(items):  
2     for item in items:  
3         for item2 in items:  
4             print(item, item2)  
5  
6 quadratic_algo([4, 5, 6, 8])
```

- Outer loop 1번에 대해 inner loop가 n번 실행되므로
- 총 스텝 수는  $n * n = n^2$
- quadratic\_algo의 시간 복잡도는 quadratic, 이는  $O(n^2)$ 으로 나타낸다

# 다양한 Big-O functions

- Logarithmic Complexity –  $O(\log N)$ 
  - 알고리즘의 실행에 필요한 스텝의 수가 인풋에 포함된 원소의 수  $N$ 에 대해  $\log N$ (밑 무관)의 형태일 때
  - 예시: 리스트의 타깃 원소의 인덱스 찾기
  - 업 다운을 생각해보자
  - 인풋 사이즈가 2배가 되어도 스텝은 하  
나만 증가!
  - 인풋 사이즈가 10에서 20이 되어도 중  
간 원소보다 큰지 작은지만 알면 나머지  
10개 중에 찾으려 된다
  - 스텝 수 =  $\log(2)N$

```
1 def binary_search(array, target, left, right):
2     middle_idx = (left+right)//2
3     print(middle_idx)
4     middle = array[middle_idx]
5     if target == middle:
6         print('answer {}'.format(middle_idx))
7     elif middle > target:
8         binary_search(array, target, left, middle_idx-1)
9     elif middle < target:
10        binary_search(array, target, middle_idx+1, right)
11    else:
12        return False
13
14 target = 25
15 m_list = [30, 94, 27, 92, 21, 37, 25, 47, 25, 53, 98, 19, 32, 32, 7]
16 length = len(m_list)
17
18 m_list = sorted(m_list)
19 left = 0
20 right = length-1
21
22 binary_search(m_list, target, 0, right)
```

# list Big - O(좌) vs. set Big - O(우)

|    | Operation                 | Example                    | Class                | Notes                  |    | Operation                 | Example                    | Class                            | Notes                             |
|----|---------------------------|----------------------------|----------------------|------------------------|----|---------------------------|----------------------------|----------------------------------|-----------------------------------|
| 1  | Index                     | <code>l[i]</code>          | $O(1)$               | 인덱스로 값 찾기              | 1  | Add                       | <code>s.add(5)</code>      | $O(1)$                           | 집합 요소 추가                          |
| 2  | Store                     | <code>l[i] = 0</code>      | $O(1)$               | 인덱스로 데이터 저장            | 2  | Containment               | <code>x in/not in s</code> | $O(1)$                           | 포함 여부 확인                          |
| 3  | Length                    | <code>len(l)</code>        | $O(1)$               | 리스트 길이                 | 3  | Remove                    | <code>s.remove(..)</code>  | $O(1)$                           | 요소 제거                             |
| 4  | Append                    | <code>l.append(5)</code>   | $O(1)$               | 리스트 뒤에 데이터 저장          | 4  | Discard                   | <code>s.discard(..)</code> | $O(1)$                           | 특정 요소 제거                          |
| 5  | Pop                       | <code>l.pop()</code>       | $O(1)$               | 가장 뒤의 데이터 pop          | 5  | Pop                       | <code>s.pop()</code>       | $O(1)$                           | 랜덤하게 하나 pop                       |
| 6  | Clear                     | <code>l.clear()</code>     | $O(1)$               | <code>l = []</code>    | 6  | Clear                     | <code>s.clear()</code>     | $O(1)$                           | similar to <code>s = set()</code> |
| 7  | Slice                     | <code>l[a:b]</code>        | $O(b-a)$             | 슬라이싱되는 요소들 수 만큼 비례     | 7  | Construction              | <code>set(...)</code>      | $O(\text{len}(...))$             | 길이만큼                              |
| 8  | Extend                    | <code>l.extend(...)</code> | $O(\text{len}(...))$ | 확장되는 길이만큼              | 8  | check <code>==, !=</code> | <code>s != t</code>        | $O(\text{len}(s))$               | 전체 요소 동일 여부 확인                    |
| 9  | Construction              | <code>list(...)</code>     | $O(\text{len}(...))$ | 리스트 길이만큼               | 9  | <code>&lt;= / &lt;</code> | <code>s &lt;= t</code>     | $O(\text{len}(s))$               | 부분집합 여부                           |
| 10 | check <code>==, !=</code> | <code>l1 == l2</code>      | $O(N)$               | 전체 리스트가 동일한지 확인        | 10 | <code>&gt;= / &gt;</code> | <code>s &gt;= t</code>     | $O(\text{len}(t))$               | 부분집합 여부                           |
| 11 | Insert                    | <code>l[a:b] = ...</code>  | $O(N)$               | 데이터 삽입                 | 11 | Union                     | <code>s, t</code>          | $O(\text{len}(s)+\text{len}(t))$ | 합집합                               |
| 12 | Delete                    | <code>del l[i]</code>      | $O(N)$               | 데이터 삭제                 | 12 | Intersection              | <code>s &amp; t</code>     | $O(\text{len}(s)+\text{len}(t))$ | 교집합                               |
| 13 | Containment               | <code>x in/not in l</code> | $O(N)$               | 포함 여부 확인               | 13 | Difference                | <code>s - t</code>         | $O(\text{len}(s)+\text{len}(t))$ | 차집합                               |
| 14 | Copy                      | <code>l.copy()</code>      | $O(N)$               | 복제                     | 14 | Symmetric Diff            | <code>s ^ t</code>         | $O(\text{len}(s)+\text{len}(t))$ | 여집합                               |
| 15 | Remove                    | <code>l.remove(...)</code> | $O(N)$               | 제거                     | 15 | Iteration                 | <code>for v in s:</code>   | $O(N)$                           | 전체 요소 순회                          |
| 16 | Pop                       | <code>l.pop(i)</code>      | $O(N)$               | 제거된 값 이후를 전부 한칸씩 당겨줘야함 | 16 | Copy                      | <code>s.copy()</code>      | $O(N)$                           | 복제                                |
| 17 | Extreme value             | <code>min(l)/max(l)</code> | $O(N)$               | 전체 데이터를 확인해야함          |    |                           |                            |                                  |                                   |
| 18 | Reverse                   | <code>l.reverse()</code>   | $O(N)$               | 뒤집기                    |    |                           |                            |                                  |                                   |
| 19 | Iteration                 | <code>for v in l:</code>   | $O(N)$               | 전체 데이터 확인하므로           |    |                           |                            |                                  |                                   |
| 20 | Sort                      | <code>l.sort()</code>      | $O(N \log N)$        | 파이썬 기본 정렬 알고리즘         |    |                           |                            |                                  |                                   |
| 21 | Multiply                  | <code>k*l</code>           | $O(k N)$             | 리스트의 곱은 리스트 개수 늘어남     |    |                           |                            |                                  |                                   |

# 딕셔너리 Big - O

- 데이터 저장, 제거, 키/값 확인, 길이 출력 등등 모두  $O(1)$ 이네!
- List 대신 딕셔너리를 많이 사용하는 이유
- List는 데이터 삽입, 제거, 포함 여부 확인 등 모두  $O(n)$
- Dict는 어떻게 작동하는 걸까?
- Hash Table? Hash Function?

|    | Operation      | Example                  | Class                | Notes                                                   |
|----|----------------|--------------------------|----------------------|---------------------------------------------------------|
| 1  | Store          | <code>d[k] = v</code>    | $O(1)$               | 데이터 저장                                                  |
| 2  | Length         | <code>len(d)</code>      | $O(1)$               | 길이 출력                                                   |
| 3  | Delete         | <code>del d[k]</code>    | $O(1)$               | 요소 제거                                                   |
| 4  | get/setdefault | <code>d.get(k)</code>    | $O(1)$               | key에 따른 value 확인                                        |
| 5  | Pop            | <code>d.pop(k)</code>    | $O(1)$               | pop                                                     |
| 6  | Pop item       | <code>d.popitem()</code> | $O(1)$               | 랜덤하게 선택해서 pop                                           |
| 7  | Clear          | <code>d.clear()</code>   | $O(1)$               | similar to <code>s = {}</code> or <code>= dict()</code> |
| 8  | View           | <code>d.keys()</code>    | $O(1)$               | same for <code>d.values()</code> / 키값 전체 확인             |
| 9  | Construction   | <code>dict(...)</code>   | $O(\text{len}(...))$ | (key, value) 튜플 개수만큼                                    |
| 10 | Iteration      | <code>for k in d:</code> | $O(N)$               | 전체 딕셔너리 순회                                              |

# 딕셔너리 Big - O

- 딕셔너리가 리스트보다 빠르다면, 딕셔너리로 리스트를 대체해볼까?

```
list_dict = {303: None, 203: None, 345: None, 348: None, 247: None, 250: None}
```

- 이미 리스트를 만들어놨지만 아이템을 추가하거나 삭제하거나 확인할 일이 많다면?
  - 딕셔너리로 변환!

```
1 entity_ids = [303, 203, 345, 348, 247, 250]
2 id_hash = {}
3 for entity_id in entity_ids:
4     id_hash.setdefault(entity_id)
5
6 print(id_hash)
```

```
{303: None, 203: None, 345: None, 348: None, 247: None, 250: None}
```

# 탐색 알고리즘

•  
수색 및 길찾기

Is this the simplest (and most surprising) sorting algorithm ever?

Stanley P. Y. Fung \*

# 정렬 알고리즘

<https://seongjaemoon.github.io/python/2017/12/16/pythonSort.html>

<http://ejklike.github.io/2017/03/04/sorting-algorithms-with-python.html>

리스트의 값들을 순서대로 정렬하는 방법은?  
리스트에서 가장 작은 수를 찾아 앞에 가져다 놓으면?



다양한 방법이 존재한다.  
빠르게 하는것이 중요한가?  
효율적으로 하는 것이 중요한가?

목적에 맞게 있는걸 가져다 쓰면 된다

| 알고리즘  | 자료구조 | 시간복잡도         |               |               | 공간복잡도    |
|-------|------|---------------|---------------|---------------|----------|
|       |      | 최선            | 평균            | 최악            |          |
| 퀵 정렬  | 배열   | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(n)$   |
| 병합 정렬 | 배열   | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$   |
| ힵ 정렬  | 배열   | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$   |
| 거품 정렬 | 배열   | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$   |
| 삽입 정렬 | 배열   | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$   |
| 선택 정렬 | 배열   | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$   |
| 버킷 정렬 | 배열   | $O(n+k)$      | $O(n+k)$      | $O(n^2)$      | $O(nk)$  |
| 기수 정렬 | 배열   | $O(nk)$       | $O(nk)$       | $O(nk)$       | $O(n+k)$ |

그런데 새로운 것이 등장?

흠.... 고민이 된다

```
[9]: import random
import copy
```

```
help(random.randint)
```

Help on method randint in module random:

randint(a, b) method of random.Random instance  
Return random integer in range [a, b], including both end points.

```
[10]: def new_sort(L):
    l = len(L)
    for i in range(l):
        for j in range(l):
            if L[i] < L[j]:
                L[i], L[j] = L[j], L[i]
    return L
```

**Algorithm 1** ICan'tBelieveItCanSort( $A[1..n]$ )  
for  $i = 1$  to  $n$  do  
  for  $j = 1$  to  $n$  do  
    if  $A[i] < A[j]$  then  
      swap  $A[i]$  and  $A[j]$

```
[11]: my_list = [random.randint(0,1000) for _ in range(10)]

old_list = copy.deepcopy(my_list)
sorted_out = new_sort(my_list)

for i,j in zip(old_list,sorted_out):
    print(f"{i} {j}")
```

```
300 266
885 300
465 378
754 444
378 465
444 625
625 707
905 754
707 885
266 905
```

## 버블 정렬

5 6 3 1 8 7 2 4

- 인접한 두 수를 비교하며 정렬해나가는 방법으로  $O(n^2)$ 의 느린 성능을 가지고 있습니다.
- 앞에서부터 시작하여 큰 수를 뒤로 보내 뒤가 가장 큰 값을 가지도록 완성해나가는 방법

```
array = [9,8,7,6,5,4,3,2,1]

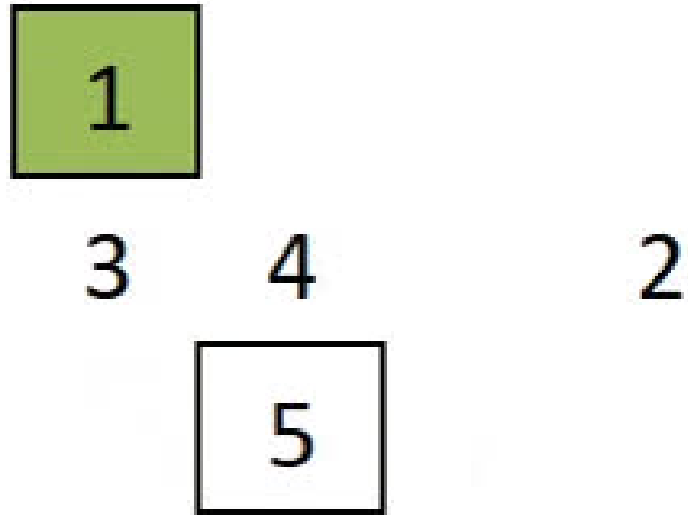
def bubble_sort(array):
    n = len(array)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if array[j] > array[j + 1]:
                array[j], array[j + 1] = array[j + 1], array[j]
        print(array)

print("before: ",array)
bubble_sort(array)
print("after:", array)
```

### 결과:

```
before:  [9, 8, 7, 6, 5, 4, 3, 2, 1]
[8, 7, 6, 5, 4, 3, 2, 1, 9]
[7, 6, 5, 4, 3, 2, 1, 8, 9]
[6, 5, 4, 3, 2, 1, 7, 8, 9]
[5, 4, 3, 2, 1, 6, 7, 8, 9]
[4, 3, 2, 1, 5, 6, 7, 8, 9]
[3, 2, 1, 4, 5, 6, 7, 8, 9]
[2, 1, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
after: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 선택정렬



## Selection Sort

- 한 바퀴 돌 때 가장 작은 값을 찾아 맨 앞과 교환하는 방식의 정렬.
- 앞에서부터 정렬해나가는 특성을 가지고 있고 버블 정렬과 마찬가지로 최악의 성능을 가진 알고리즘입니다.

```
array = [8,4,6,2,9,1,3,7,5]
```

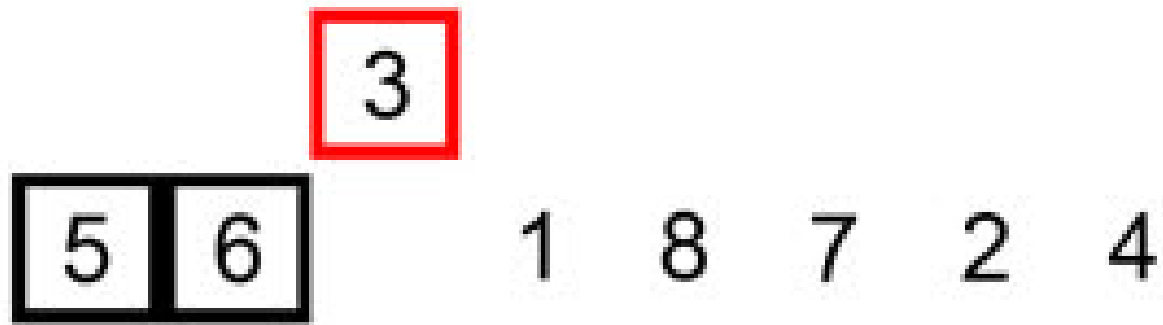
```
def selection_sort(array):  
    n = len(array)  
    for i in range(n):  
        min_index = i  
        for j in range(i + 1, n):  
            if array[j] < array[min_index]:  
                min_index = j  
        array[i], array[min_index] = array[min_index], array[i]  
        print(array[:i+1])
```

```
print("before: ",array)  
selection_sort(array)  
print("after:", array)
```

## 결과:

```
before: [8, 4, 6, 2, 9, 1, 3, 7, 5]  
[1]  
[1, 2]  
[1, 2, 3]  
[1, 2, 3, 4]  
[1, 2, 3, 4, 5]  
[1, 2, 3, 4, 5, 6]  
[1, 2, 3, 4, 5, 6, 7]  
[1, 2, 3, 4, 5, 6, 7, 8]  
[1, 2, 3, 4, 5, 6, 7, 8, 9]  
after: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 삽입정렬



- 정렬된 데이터 그룹을 늘려가며 추가되는 데이터는 알맞은 자리에 삽입하는 방식
- 버블 정렬과 선택 정렬과 마찬가지로 최악의 성능인 정렬 알고리즘입니다.

```
array = [8,4,6,2,9,1,3,7,5]

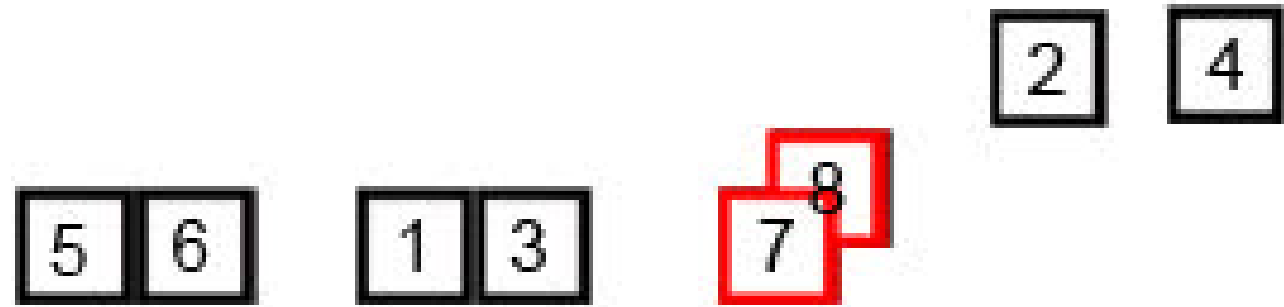
def insertion_sort(array):
    n = len(array)
    for i in range(1, n):
        for j in range(i, 0, - 1):
            if array[j - 1] > array[j]:
                array[j - 1], array[j] = array[j], array[j - 1]
        print(array[:i+1])

print("before: ",array)
insertion_sort(array)
print("after:", array)
```

결과:

```
before:  [8, 4, 6, 2, 9, 1, 3, 7, 5]
[4, 8]
[4, 6, 8]
[2, 4, 6, 8]
[2, 4, 6, 8, 9]
[1, 2, 4, 6, 8, 9]
[1, 2, 3, 4, 6, 8, 9]
[1, 2, 3, 4, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
after: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 병합정렬



- 분할 정복과 재귀를 이용한 알고리즘으로  $O(n \log n)$ 의 시간 복잡도를 가지고 있어 괜찮은 성능을 보여 줍니다.
- 반으로 쪼개고 다시 합치는 과정에서 그룹을 만들어 정렬하게 되며 이 과정에서  $2n$ 개의 공간이 필요합니다.
- 합쳐지는 과정은 다음과 같습니다.

```
array = [8,4,6,2,9,1,3,7,5]
```

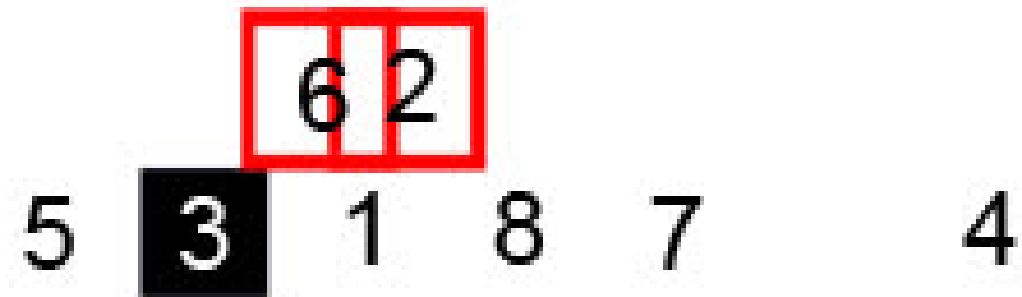
```
def merge_sort(array):
    if len(array) < 2:
        return array
    mid = len(array) // 2
    low_arr = merge_sort(array[:mid])
    high_arr = merge_sort(array[mid:])

    merged_arr = []
    l = h = 0
    while l < len(low_arr) and h < len(high_arr):
        if low_arr[l] < high_arr[h]:
            merged_arr.append(low_arr[l])
            l += 1
        else:
            merged_arr.append(high_arr[h])
            h += 1
    merged_arr += low_arr[l:]
    merged_arr += high_arr[h:]
    print(merged_arr)
    return merged_arr
```

```
print("before: ",array)
array = merge_sort(array)
print("after:", array)
```

```
before: [8, 4, 6, 2, 9, 1, 3, 7, 5]
[4, 8]
[2, 6]
[2, 4, 6, 8]
[1, 9]
[5, 7]
[3, 5, 7]
[1, 3, 5, 7, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
after: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 퀵 정렬



- 퀵 정렬은 Pivot을 설정하고 그 기준으로 정렬을 합니다. 빠르고 추가적인 메모리 소모가 없음
- 다른 정렬 알고리즘보다 빠르며 많은 언어의 정렬 내장 함수로 퀵 정렬을 수행합니다.

```
array = [8,4,6,2,5,1,3,7,9]
```

```
def quick_sort(array):  
    if len(array) <= 1:  
        return array  
    pivot = len(array) // 2  
    front_arr, pivot_arr, back_arr = [], [], []  
    for value in array:  
        if value < array[pivot]:  
            front_arr.append(value)  
        elif value > array[pivot]:  
            back_arr.append(value)  
        else:  
            pivot_arr.append(value)  
    print(front_arr, pivot_arr, back_arr)  
    return quick_sort(front_arr) + quick_sort(pivot_arr) + quick_sort(back_arr)
```

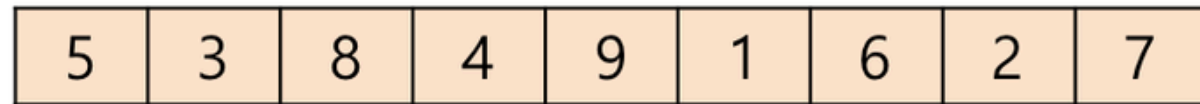
```
print("before: ",array)  
array = quick_sort(array)  
print("after:", array)
```

피벗 기준 분할 과정:

```
[4, 2, 1, 3] [5] [8, 6, 7, 9]  
[] [1] [4, 2, 3]  
[] [2] [4, 3]  
[] [3] [4]  
[6] [7] [8, 9]  
[8] [9] []  
after: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 퀵 정렬의 이해

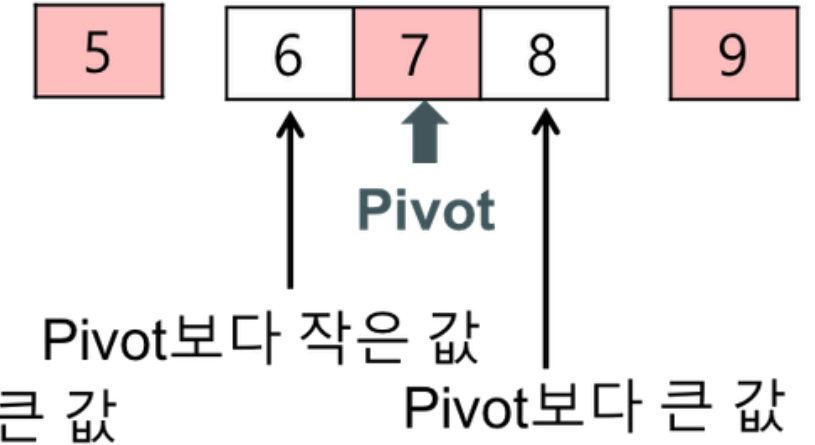
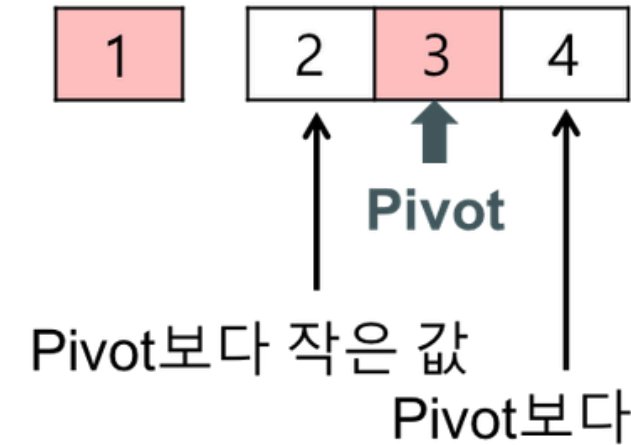
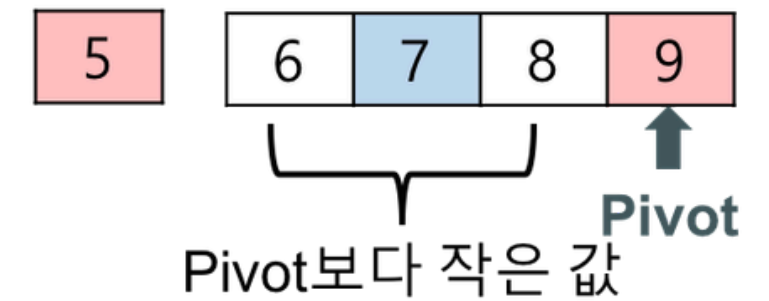
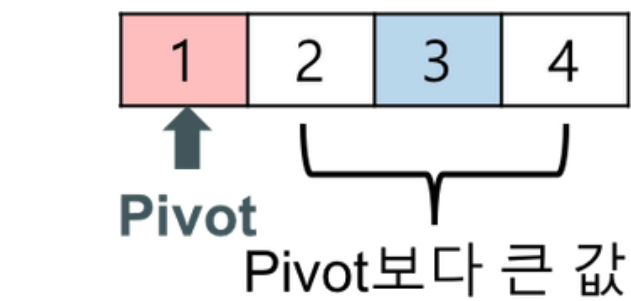
초기상태



Pivot보다 작은 값

Pivot

Pivot보다 큰 값



리스트의 크기가 0이나 1이 될 때까지 반복



오름차순  
완성상태



# 자료구조

큐, 스택, 트리

## 자료구조? 자료형?

- Number, String
- Tuple, List
- Dictionary, Set
- 어디에 쓰는거지?



# 자료구조?

- 그게 뭐죠?
  - 먼저 알아야 할 개념
    - ADT
    - Stack(팬케이크 자료 구조)
    - Queue(줄 서기)
    - 재귀 함수(복습)



# Stack과 Queue

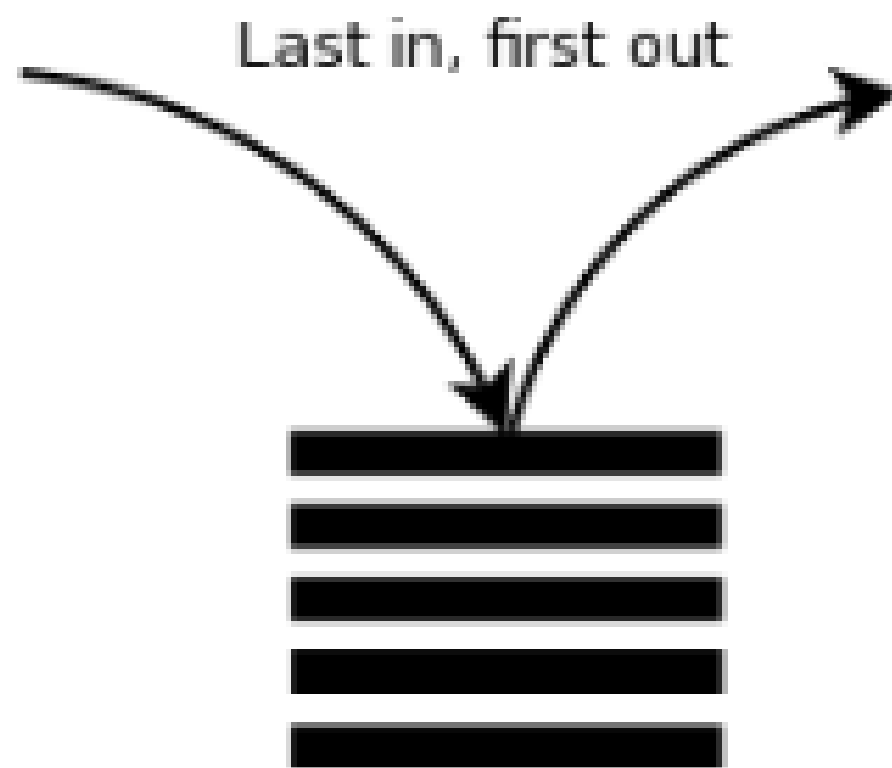
---

- Abstract Data Type/Structure(ADT)
  - 자료 구조의 규칙으로 둘 중 하나로 구분된다
    - Stack
    - Queue
  - 실존하지 않는 추상적 양식으로, 구조의 행동 양식만 정의되어 있다

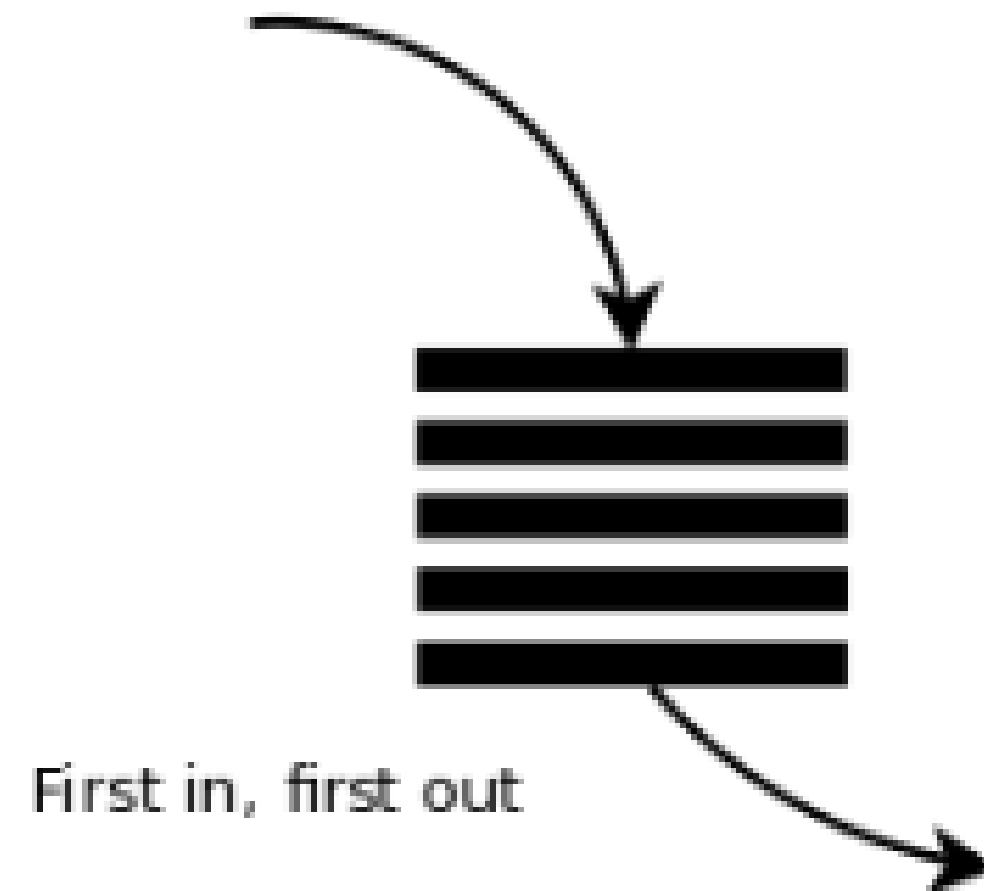
# Stack vs Queue

---

**Stack:**



**Queue:**



# Stack과 Queue

---

- Stack
  - 후입선출(LIFO – Last In, First Out)
  - 팬케이크 자료 구조(새로 구운 건 위에 쌓고, 먹을 땐 위에 것부터)
  - 요소의 추가와 삭제, 읽기를 맨 위에서부터만 할 수 있음
  - 예시:
    - 웹 브라우저에서 뒤로 가기 버튼
      - 웹페이지 히스토리 스택의 맨 위에서 한 페이지를 가져가는 것
    - ctrl + z
      - 유저가 하고 있는 행동을 스택에 차곡차곡 쌓다가 ctrl + z 하면 스택으로 가서 차례차례 가져오는 것

# Stack과 Queue

---

- Stack의 활용
  - 스택은 list 자료형으로 구현 가능, 가장 오른쪽 끝에 원소를 삽입하는 append와 오른쪽 끝에서 원소를 꺼내는 pop 메서드가 있음
  - 큐에서와 달리 list 기본 메서드로  $O(1)$ 로 구현 가능
    - `pop("원소")` 는  $O(N)$ 이고 `pop()`은  $O(1)$
    - 전자는 특정 값 제거 후 그 다음 값들을 전부 앞으로 한 칸씩 당겨줘야 함
    - 후자는 가장 뒤의 데이터를 꺼냄
    - 전자를 사용하지 않고 후자를 stack에서 사용하는 경우 별도 라이브러리 활용 없이 기본 자료형과 메서드만으로 구현 가능

# Stack과 Queue

---

- Stack의 활용

```
1
2
3 stack = []
4
5 # 삽입(5) - 삽입(2) - 삽입(3) - 삽입(7) - 삭제() - 삽입(1) - 삽입(4) - 삭제()
6 stack.append(5)
7 stack.append(2)
8 stack.append(3)
9 stack.append(7)
10 stack.pop()
11 stack.append(1)
12 stack.append(4)
13 stack.pop()
14
15 print(stack) # 최하단 원소부터 출력
16 print(stack[::-1]) # 최상단 원소부터 출력
```

# Stack과 Queue

---

- Queue
  - 줄 서기로 이해하면 좋다
  - 선입선출(Fifo - first in, first out)
  - 새로운 요소는 맨 뒤에 추가됨
  - 큐의 맨 앞에 있는 요소만 읽거나 삭제될 수 있음
  - 예시:
    - 이메일 전달
    - 푸쉬 알림
    - 쇼핑몰 주문 처리
    - 콜센터

# Stack과 Queue

- Queue의 활용
  - 리스트만으로도 구현이 가능하지만 그럴 경우 시간복잡도가  $O(n)$ 이다
  - 따라서 collections 라이브러리의 deque로 대체하는 것을 추천
    - append는 list에서의 append와 동일하게 오른쪽 끝에 하나 붙이는 거 :  $O(1)$
    - popleft는 가장 왼쪽 데이터 꺼내기 :  $O(1)$

```
2
3 # 큐(Queue) 구현을 위해 deque 라이브러리 사용
4 queue = deque()
5
6 # 삽입(5) - 삽입(2) - 삽입(3) - 삽입(7) - 삭제() - 삽입(1) - 삽입(4) - 삭제()
7 queue.append(5)
8 queue.append(2)
9 queue.append(3)
10 queue.append(7)
11 queue.popleft()
12 queue.append(1)
13 queue.append(4)
14 queue.popleft()
15
16 print(queue) # 먼저 들어온 순서대로 출력
17 queue.reverse() # 다음 출력을 위해 역순으로 바꾸기
18 print(queue) # 나중에 들어온 원소부터 출력
```

# Stack과 Queue

---

- 재귀 함수
  - DFS 구현에 사용되기도 한다
  - 마지막에 호출된 애가 실행되고 그 다음 호출한 애가 실행되고(=스택)
  - 재귀 함수는 모두 반복문을 이용하여 동일하게 구현 가능(뭐가 더 나은지는 케바케)
  - 컴퓨터가 함수를 연속적으로 호출하면 컴퓨터 메모리 내부의 스택 프레임에 쌓인다
    - 그래서 스택을 사용해야 할 때 스택 라이브러리 대신에 재귀 함수를 이용하는 경우가 많다

# Stack과 Queue

## 재귀 함수와 스택

```
1 # 재귀함수 - 스택 프레임과 유사
2
3 def recursive_function(i):
4     # 100번째 호출을 했을 때 종료되도록 종료 조건 명시
5     if i == 10:
6         return
7     print(i, '번째 재귀함수에서', i + 1, '번째 재귀함수를 호출합니다.')
8     recursive_function(i + 1)
9     print(i, '번째 재귀함수를 종료합니다.')
10
11 recursive_function(1)
```

```
1 번째 재귀함수에서 2 번째 재귀함수를 호출합니다.
2 번째 재귀함수에서 3 번째 재귀함수를 호출합니다.
3 번째 재귀함수에서 4 번째 재귀함수를 호출합니다.
4 번째 재귀함수에서 5 번째 재귀함수를 호출합니다.
5 번째 재귀함수에서 6 번째 재귀함수를 호출합니다.
6 번째 재귀함수에서 7 번째 재귀함수를 호출합니다.
7 번째 재귀함수에서 8 번째 재귀함수를 호출합니다.
8 번째 재귀함수에서 9 번째 재귀함수를 호출합니다.
9 번째 재귀함수에서 10 번째 재귀함수를 호출합니다.
9 번째 재귀함수를 종료합니다.
8 번째 재귀함수를 종료합니다.
7 번째 재귀함수를 종료합니다.
6 번째 재귀함수를 종료합니다.
5 번째 재귀함수를 종료합니다.
4 번째 재귀함수를 종료합니다.
3 번째 재귀함수를 종료합니다.
2 번째 재귀함수를 종료합니다.
1 번째 재귀함수를 종료합니다.
```

# Stack과 Queue

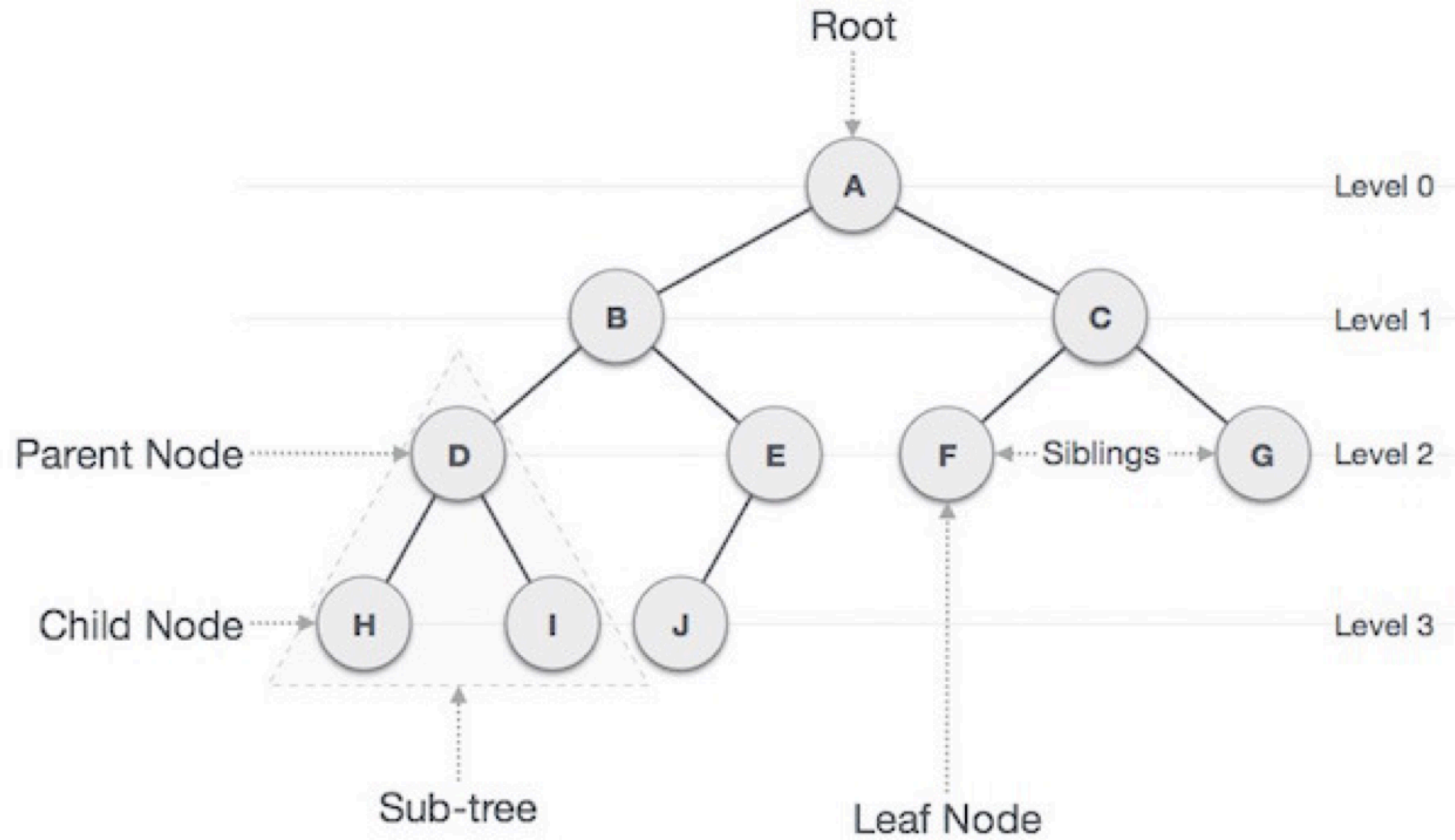
## 재귀 함수와 반복문

- 반복문을 쓸 수도, 재귀 함수를 쓸 수도
- 상황에 따라 무엇이 더 나을지를 고민하자

```
1 # 팩토리얼 예제
2 # 반복적으로 구현한 n!
3 def factorial_iterative(n):
4     result = 1
5     # 1부터 n까지의 수를 차례대로 곱하기
6     for i in range(1, n + 1):
7         result *= i
8     return result
9
10 # 재귀적으로 구현한 n!
11 def factorial_recursive(n):
12     if n <= 1: # n이 1 이하인 경우 1을 반환
13         return 1
14     # n! = n * (n - 1)!를 그대로 코드로 작성하기
15     return n * factorial_recursive(n - 1)
16
17 # 각각의 방식으로 구현한 n! 출력(n = 5)
18 print(f'반복적으로 구현: {factorial_iterative(5)}')
19 print(f'재귀적으로 구현: {factorial_recursive(5)}')
```

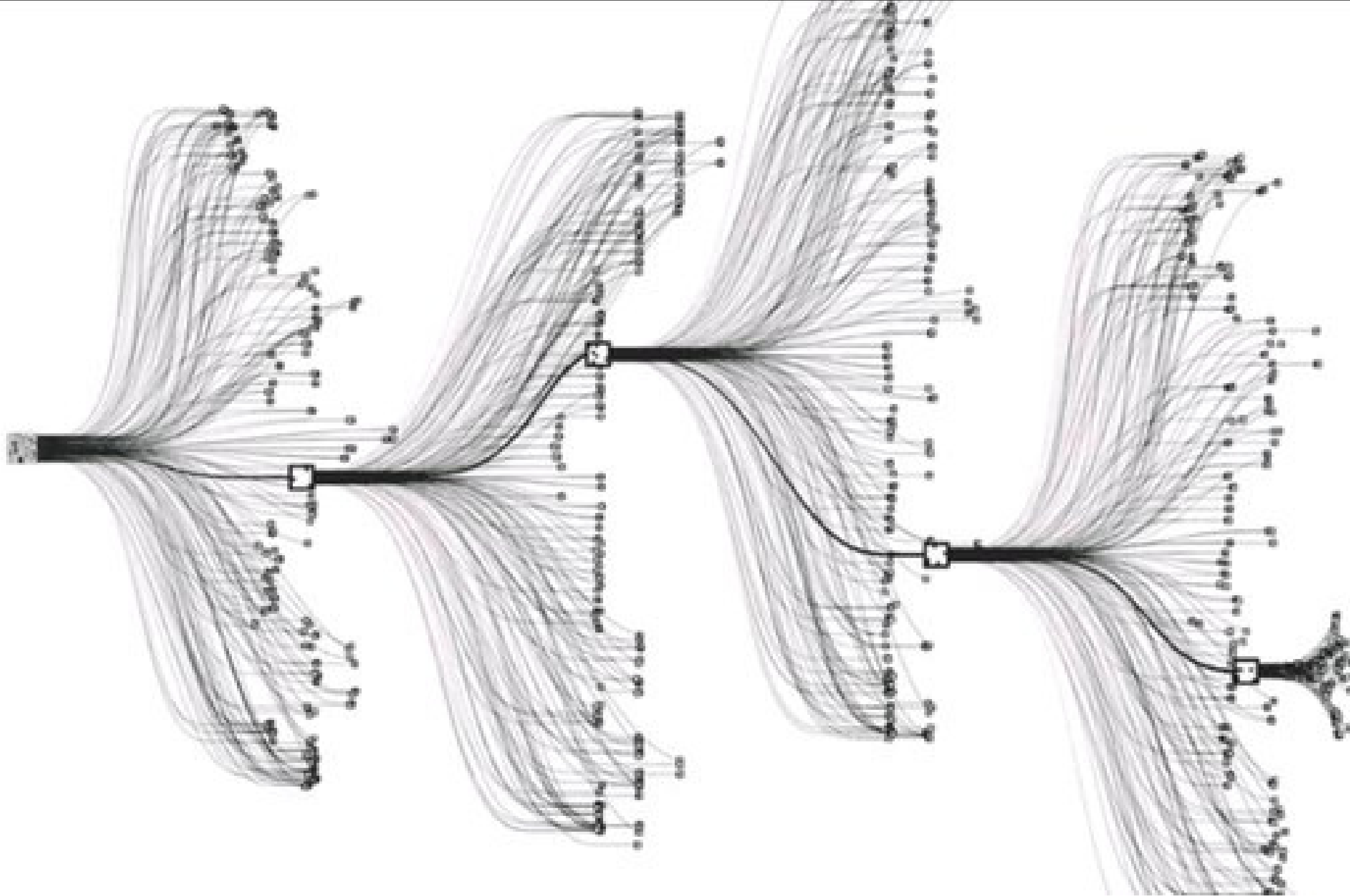
```
반복적으로 구현: 120
재귀적으로 구현: 120
```

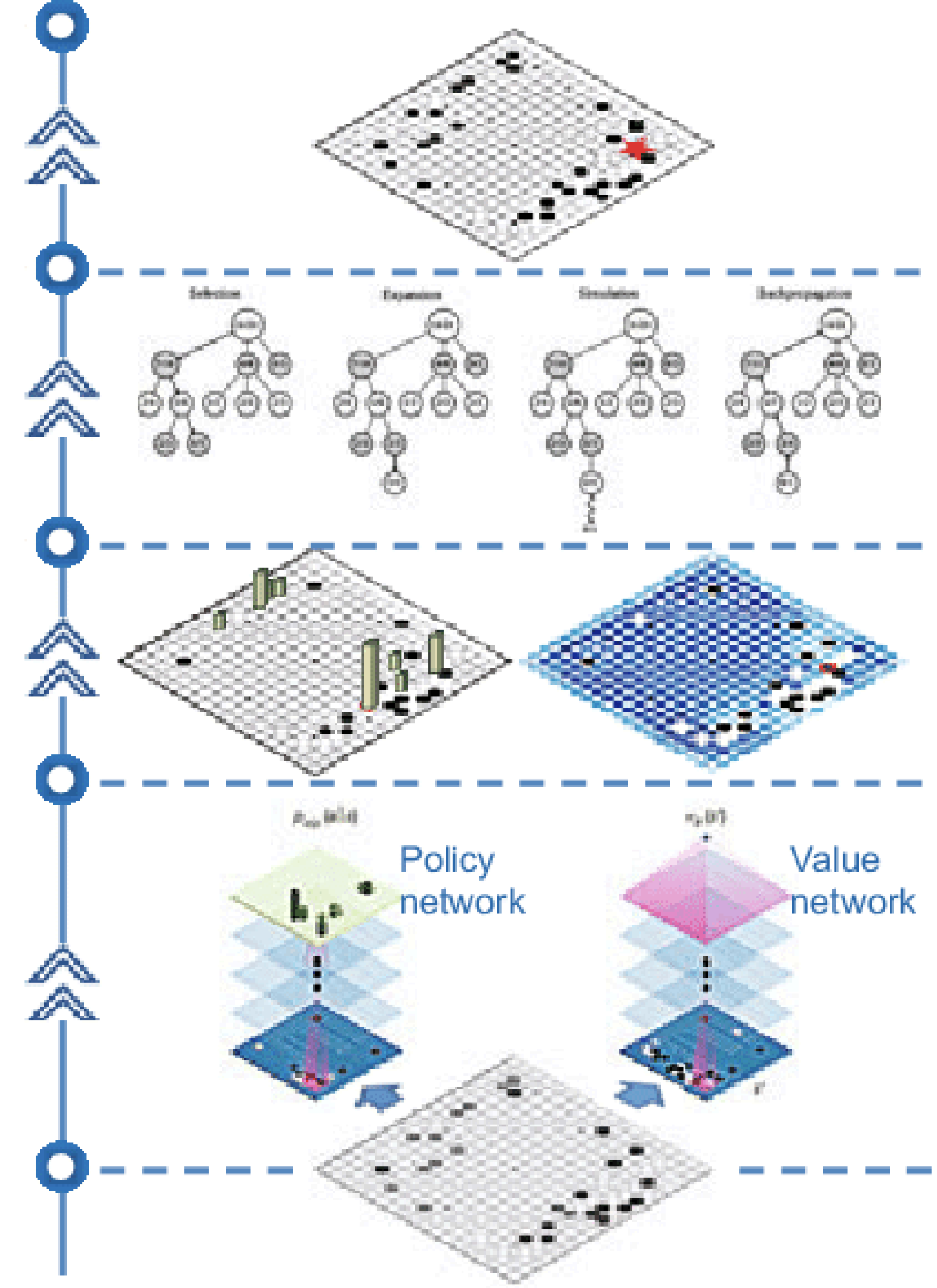
# Tree



# Tree?

---





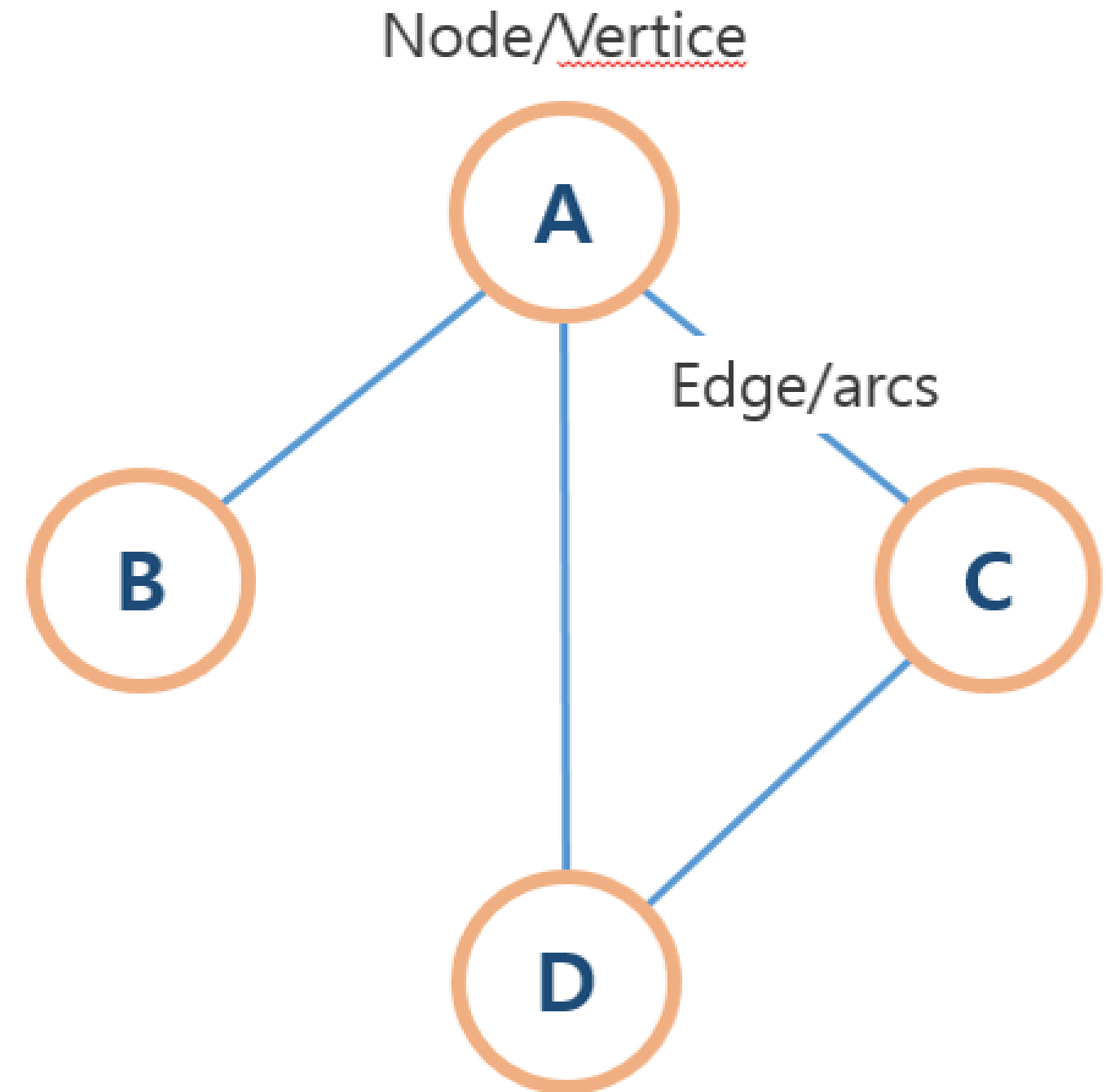
# 최단 경로 알고리즘

• A\* 알고리즘

# 최단 경로 알고리즘들

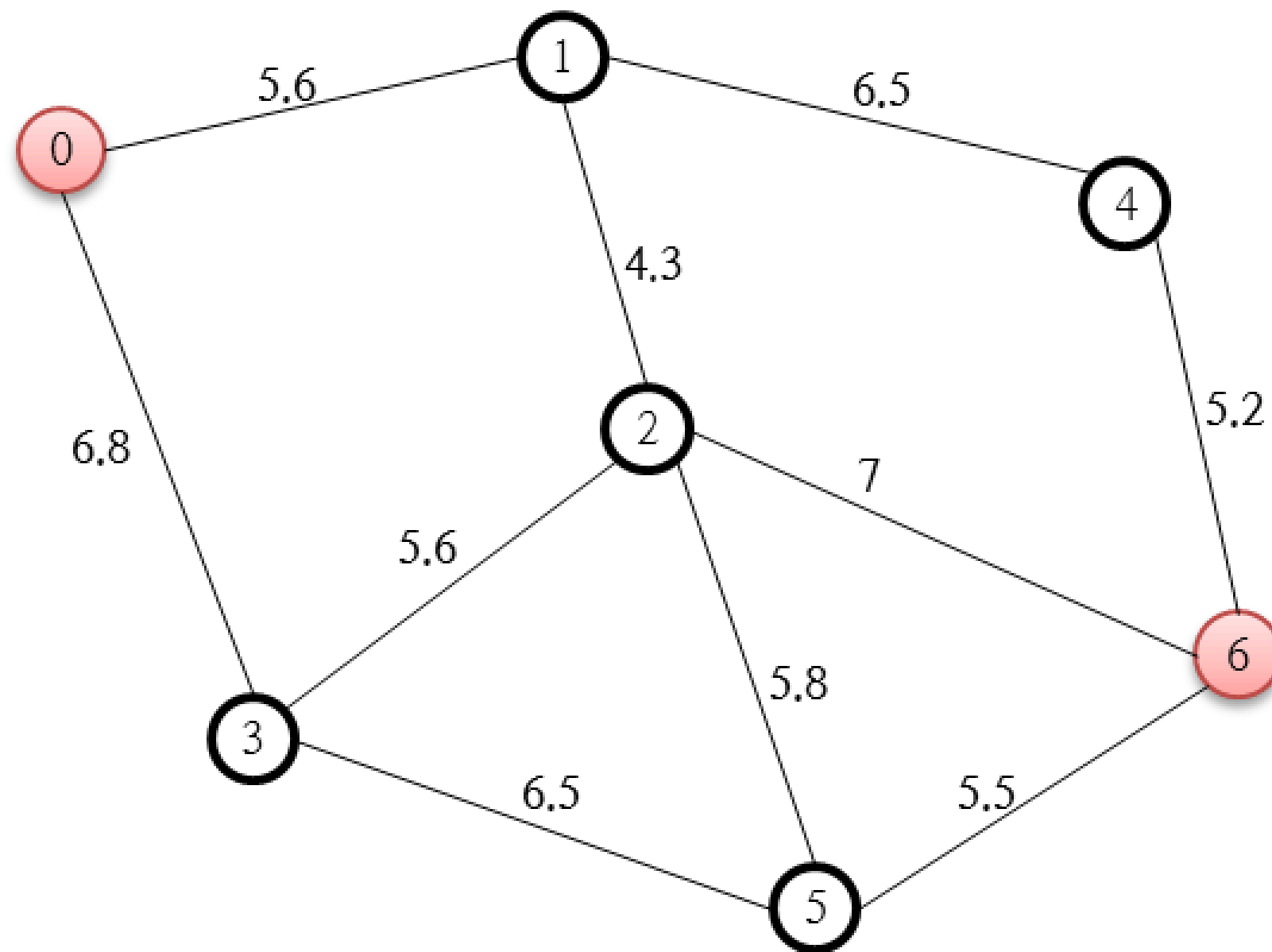
정점이랑 간선이 뭐예요

- 그래프 기본 용어
  - 그래프는 정점과 간선의 집합이다
  - 정점(Vertex, Vertices): 노드
  - 간선(Edge): 두 개의 노드들을 잇는 선



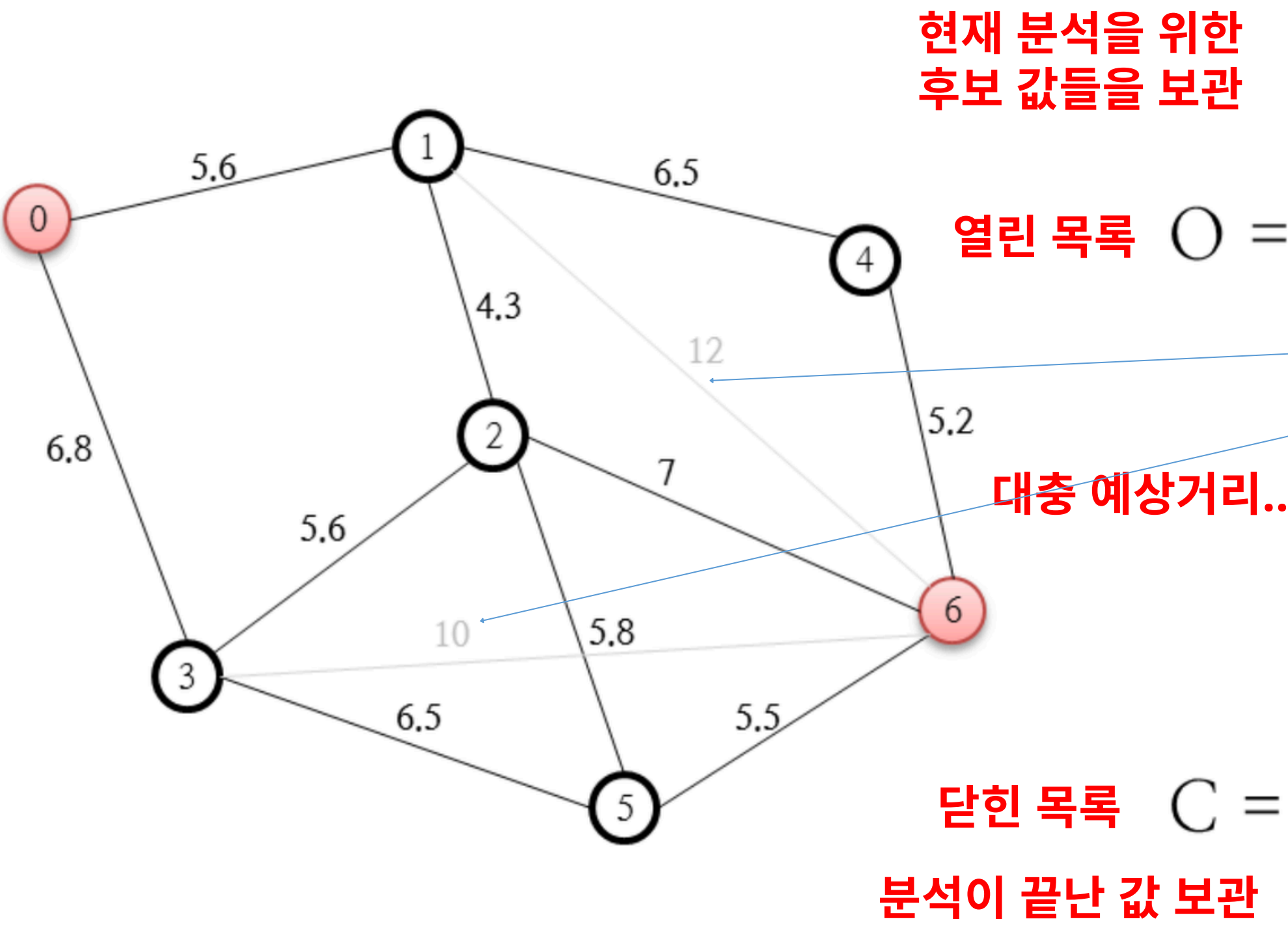
<https://velog.io/@gimtommang11/자료구조그래프>

# A\* algorithm



- 각 노드들의 번호: 0~6
- 사이의 거리: 0~1 = 5.6
- 시작 0 - 끝 6 최단 경로 찾기!

# A\* algorithm



|             |      |      |
|-------------|------|------|
| Node ID     | 1    | 3    |
| F Score     | 17.6 | 16.8 |
| G Score     | 5.6  | 6.8  |
| H Score     | 12   | 10   |
| Parent Node | 0    | 0    |

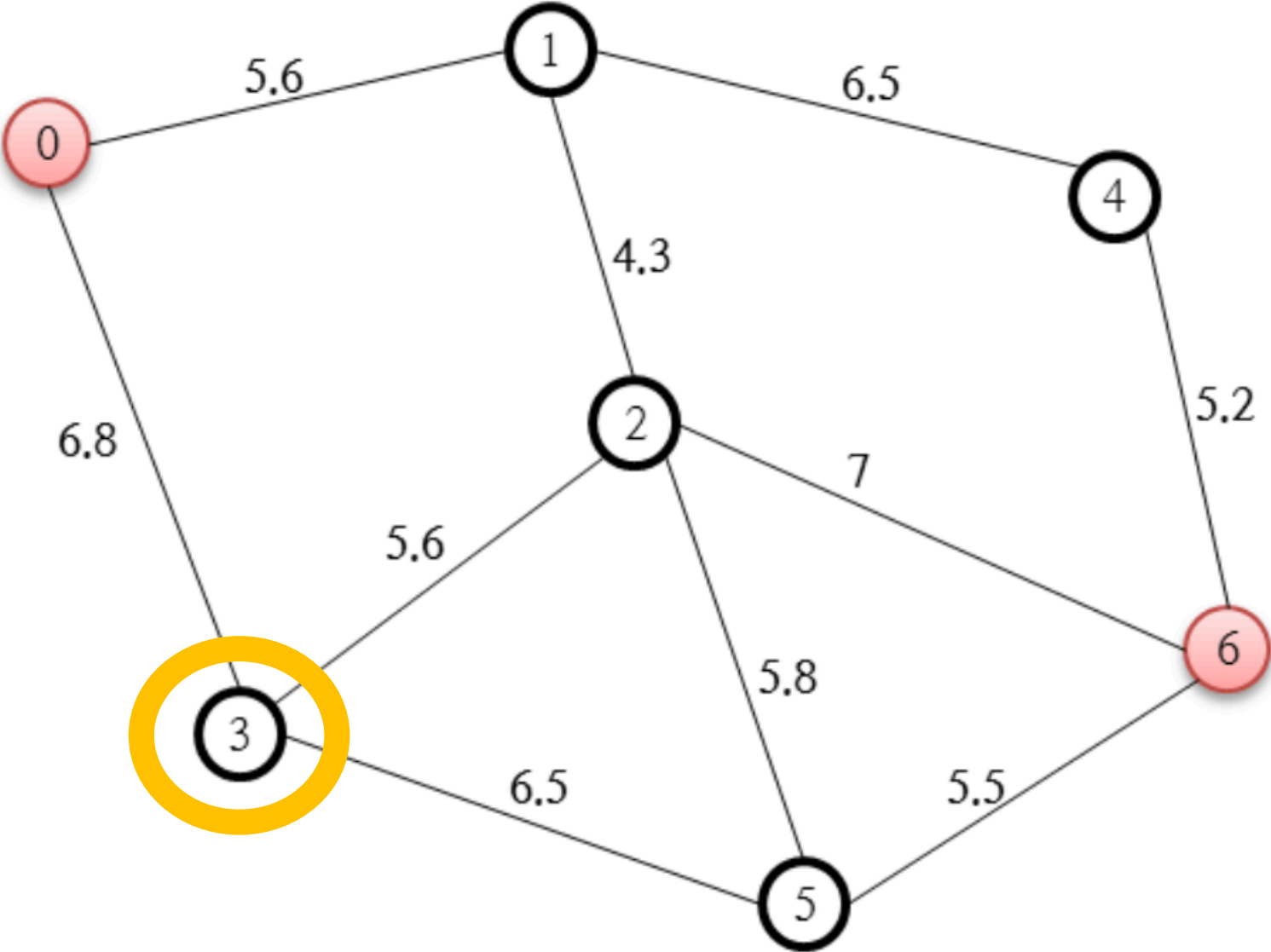
출발(0)에 붙은 2개 넣어 둬

둘 중에 F (점수합) 이 제일 낮은 것은?

|             |   |
|-------------|---|
| Node ID     | 0 |
| F Score     | 0 |
| G Score     | 0 |
| H Score     | 0 |
| Parent Node | - |

ID: 자기 노드 번호  
F = G+H  
G: 거리  
H: 최종까지 예상 거리  
Parent: 이전 노드

# A\* algorithm



O =

| Node ID     | 1    | 2    | 5    |
|-------------|------|------|------|
| F Score     | 17.6 | 19.4 | 18.8 |
| G Score     | 5.6  | 12.4 | 13.3 |
| H Score     | 12   | 7    | 5.5  |
| Parent Node | 0    | 3    | 3    |

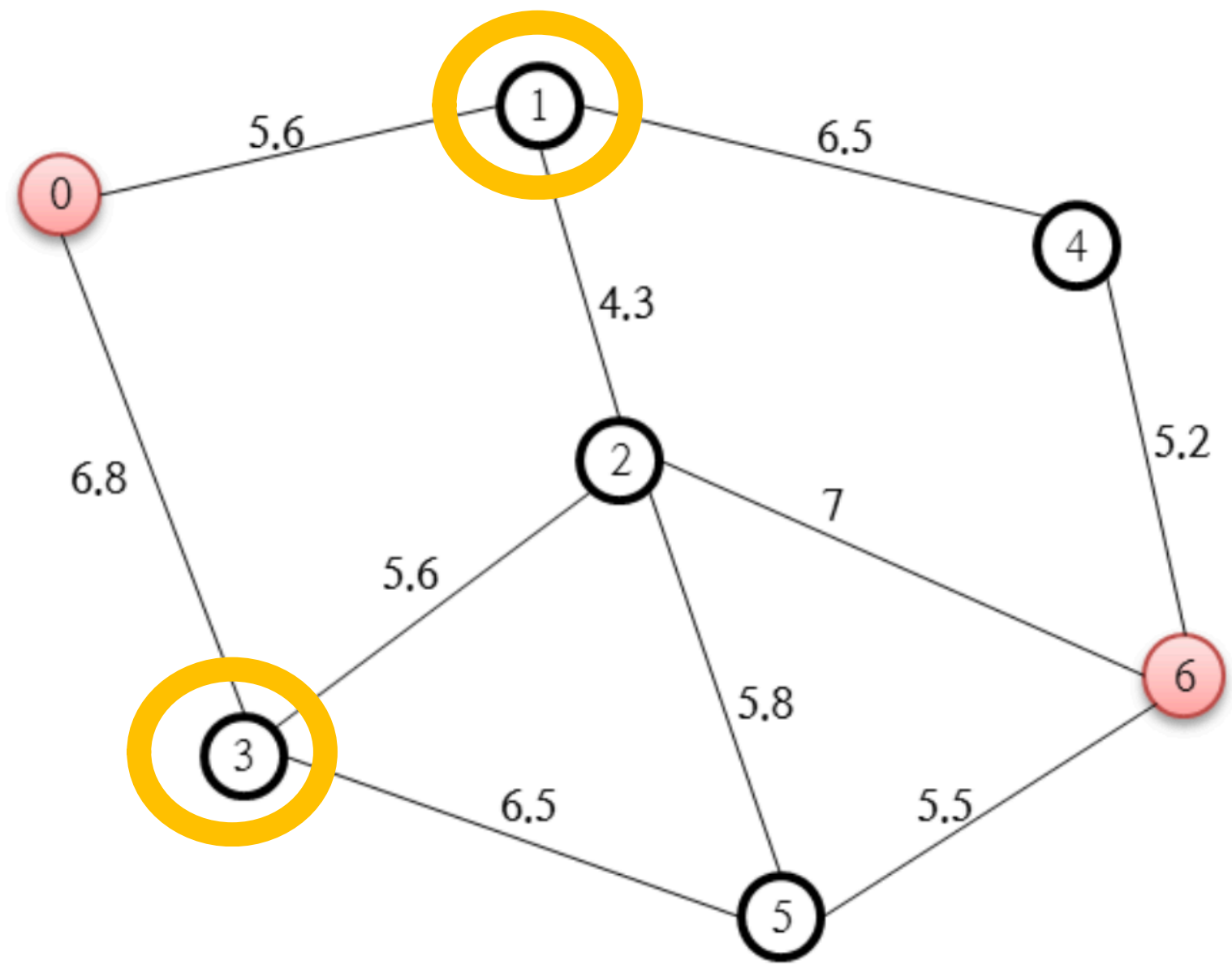
3을 옮기면서  
3옆의 2와 5를  
후보에 등록

C =

| Node ID     | 0 | 3    |
|-------------|---|------|
| F Score     | 0 | 16.8 |
| G Score     | 0 | 6.8  |
| H Score     | 0 | 10   |
| Parent Node | - | 0    |

O에서 F값이 제일 낮은  
노드 3을 C로 옮김

# A\* algorithm



|      |
|------|
| 2    |
| 19.4 |
| 12.4 |
| 7    |
| 3    |

0-3-2 보다 0-1-2가 F가 낮으니 같음!

O =

| Node ID     | 2    | 5    | 4    |
|-------------|------|------|------|
| F Score     | 16.9 | 18.8 | 17.3 |
| G Score     | 9.9  | 13.3 | 12.1 |
| H Score     | 7    | 5.5  | 5.2  |
| Parent Node | 1    | 3    | 1    |

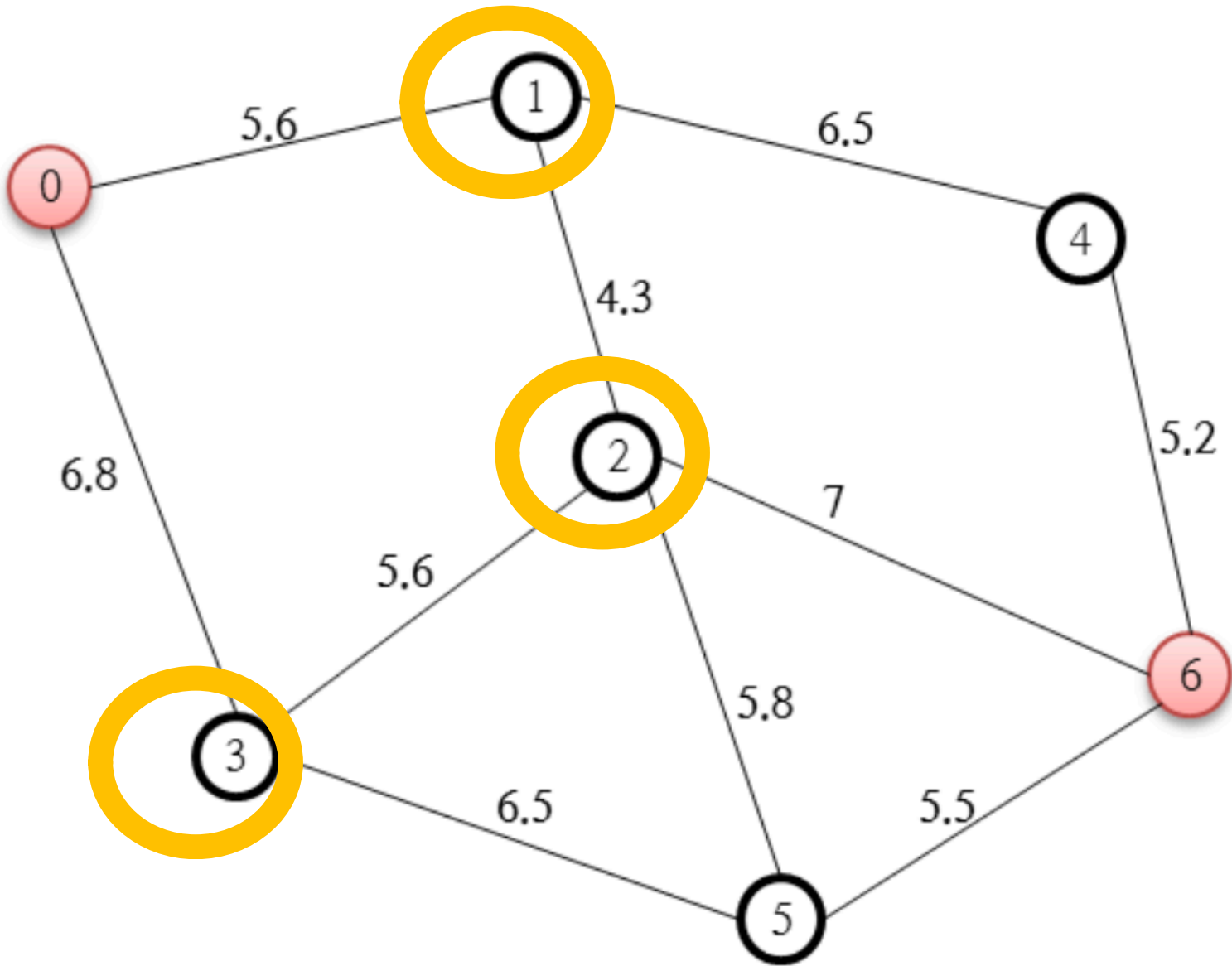
1을 옮기면서  
1옆의 2와 4를  
후보 등록 및 비교.

C =

| Node ID     | 0 | 3    | 1    |
|-------------|---|------|------|
| F Score     | 0 | 16.8 | 17.6 |
| G Score     | 0 | 6.8  | 5.6  |
| H Score     | 0 | 10   | 12   |
| Parent Node | - | 0    | 0    |

O에서 F값이 제일 낮은  
노드 1을 C로 옮김

# A\* algorithm



O =

| Node ID     | 5    | 4    | 6    |
|-------------|------|------|------|
| F Score     | 18.8 | 17.3 | 16.9 |
| G Score     | 13.3 | 12.1 | 16.9 |
| H Score     | 5.5  | 5.2  | 0    |
| Parent Node | 3    | 1    | 2    |

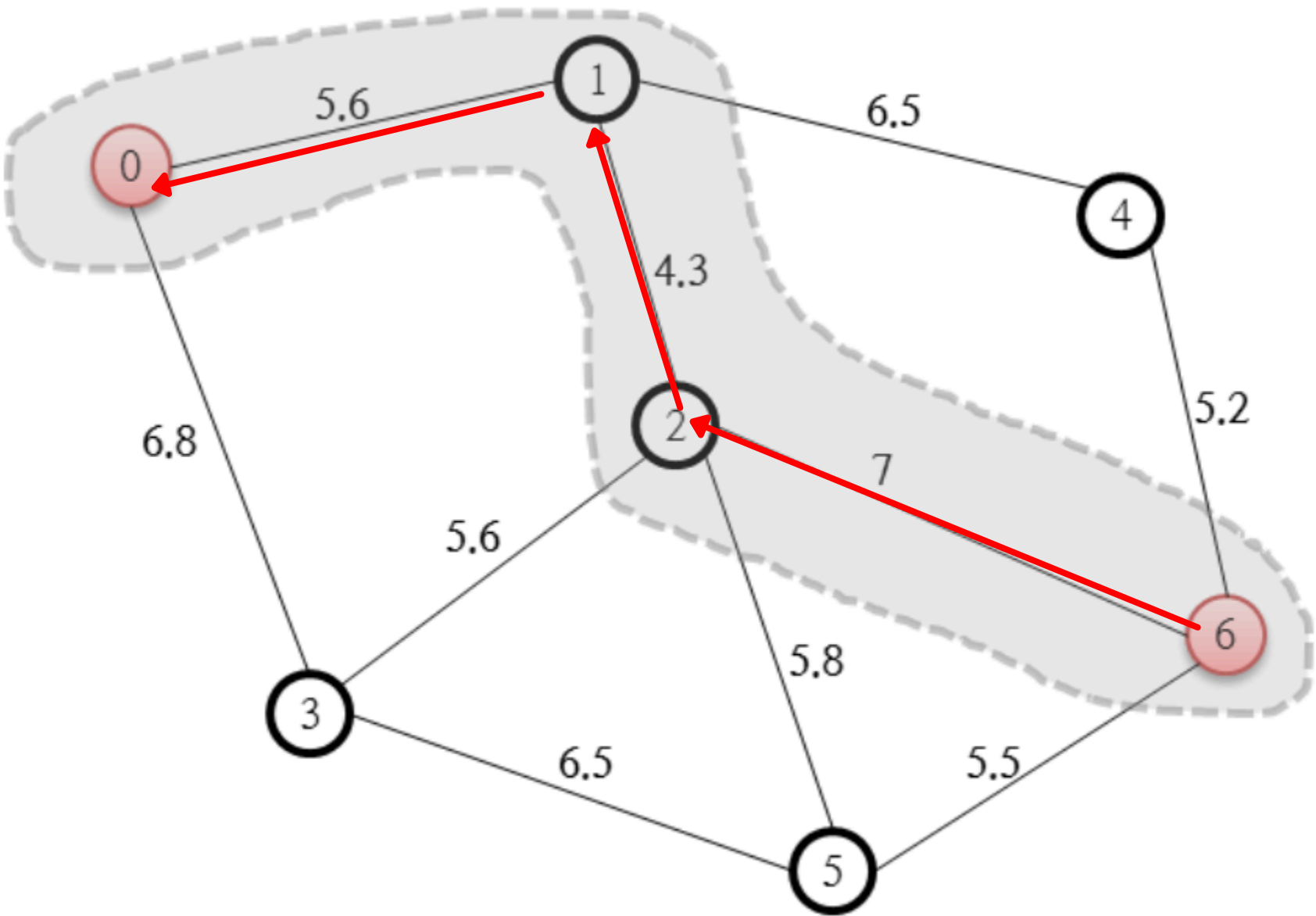
2을 옮기면서  
2옆의 6 (최종목표)  
후보 등록.

C =

| Node ID     | 0 | 3    | 1    | 2    |
|-------------|---|------|------|------|
| F Score     | 0 | 16.8 | 17.6 | 16.9 |
| G Score     | 0 | 6.8  | 5.6  | 9.9  |
| H Score     | 0 | 10   | 12   | 7    |
| Parent Node | - | 0    | 0    | 1    |

노드 2를  
C로 옮김

# A\* algorithm



O =

| Node ID     | 5    | 4    |
|-------------|------|------|
| F Score     | 18.8 | 17.3 |
| G Score     | 13.3 | 12.1 |
| H Score     | 5.5  | 5.2  |
| Parent Node | 3    | 1    |

최종지점에 도착. 종료

C =

| Node ID     | 0 | 3    | 1    | 2    | 6    |
|-------------|---|------|------|------|------|
| F Score     | 0 | 16.8 | 17.6 | 16.9 | 16.9 |
| G Score     | 0 | 6.8  | 5.6  | 9.9  | 16.9 |
| H Score     | 0 | 10   | 12   | 7    | 0    |
| Parent Node | - | 0    | 0    | 1    | 2    |

노드 6를 C로 옮김

# A\* algorithm

---

- 길 찾는 하나의 방법론
- 자료형이란?
  - 변수의 종류
  - 변수란?
    - 데이터를 저장하기 위한 저장공간
    - 저장을 왜 하는가?
      - 원하는 것을 알기 위하여
      - 알고리즘이란?

수학과 컴퓨터 과학, 언어학 또는 관련 분야에서  
어떠한 문제를 해결하기 위해 정해진  
일련의 절차나 방법을 공식화한 형태로  
표현한 것.

알고리즘의 정의 - 출처 : 위키백과

# A\* 알고리즘 코드를 보자

부제: 자료를 영어로 찾아야 할까 한글로 찾아야 할까?

```
from collections import deque
```

```
class Graph:
```

```
    # example of adjacency list (or rather map)
```

```
    # adjacency_list = {
```

```
    # 'A': [('B', 1), ('C', 3), ('D', 7)],
```

```
    # 'B': [('D', 5)],
```

```
    # 'C': [('D', 12)]
```

```
    # }
```

```
    def __init__(self, adjacency_list):
```

```
        self.adjacency_list = adjacency_list
```

```
    def get_neighbors(self, v):
```

```
        return self.adjacency_list[v]
```

```
    # heuristic function with equal values for all nodes
```

```
    def h(self, n):
```

```
        H = {
```

```
            'A': 1,
```

```
            'B': 1,
```

```
            'C': 1,
```

```
            'D': 1
```

```
        }
```

```
        return H[n]
```

<https://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/a-star-search-algorithm/>

- 인접 노드들의 리스트 예시가 주석으로 달려 있다
- 인접 리스트(딕셔너리지만)를 선언할 때 인자로 넣어줘야 한다
- get\_neighbors 함수에 특정 정점(vertex이므로 v)을 넣어주면 인접 딕셔너리에서 해당 정점을 키로 하여 그 값인 인접 정점과 그 거리를 가져온다
- h 함수는  $f = g + h$ 에서 h 값을 임의의 1이라는 값으로 고정해둔 예제용 함수

```

def a_star_algorithm(self, start_node, stop_node):
    # open_list is a list of nodes which have been visited, but who's neighbors
    # haven't all been inspected, starts off with the start node
    # closed_list is a list of nodes which have been visited
    # and who's neighbors have been inspected
    open_list = set([start_node])
    closed_list = set([])

    # g contains current distances from start_node to all other nodes
    # the default value (if it's not found in the map) is +infinity
    g = {}

    g[start_node] = 0

    # parents contains an adjacency map of all nodes
    parents = {}
    parents[start_node] = start_node

    while len(open_list) > 0:
        n = None

        # find a node with the lowest value of f() - evaluation function
        for v in open_list:
            if n == None or g[v] + self.h(v) < g[n] + self.h(n):
                n = v;

        if n == None:
            print('Path does not exist!')
            return None

```

```

        if n == None:
            print('Path does not exist!')
            return None

        # if the current node is the stop_node
        # then we begin reconstructin the path from it to the start_node
        if n == stop_node:
            reconst_path = []

            while parents[n] != n:
                reconst_path.append(n)
                n = parents[n]

            reconst_path.append(start_node)

            reconst_path.reverse()

            print('Path found: {}'.format(reconst_path))
            return reconst_path

        # for all neighbors of the current node do
        for (m, weight) in self.get_neighbors(n):
            # if the current node isn't in both open_list and closed_list
            # add it to open_list and note n as it's parent
            if m not in open_list and m not in closed_list:
                open_list.add(m)
                parents[m] = n
                g[m] = g[n] + weight

            # otherwise, check if it's quicker to first visit n, then m
            # and if it is, update parent data and g data
            # and if the node was in the closed_list, move it to open_list

```

```

# for all neighbors of the current node do
for (m, weight) in self.get_neighbors(n):
    # if the current node isn't in both open_list and closed_list
    # add it to open_list and note n as it's parent
    if m not in open_list and m not in closed_list:
        open_list.add(m)
        parents[m] = n
        g[m] = g[n] + weight

    # otherwise, check if it's quicker to first visit n, then m
    # and if it is, update parent data and g data
    # and if the node was in the closed_list, move it to open_list
    else:
        if g[m] > g[n] + weight:
            g[m] = g[n] + weight
            parents[m] = n

            if m in closed_list:
                closed_list.remove(m)
                open_list.add(m)

    # remove n from the open_list, and add it to closed_list
    # because all of his neighbors were inspected
    open_list.remove(n)
    closed_list.add(n)

print('Path does not exist!')
return None

```

<https://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/a-star-search-algorithm/>

We run the code as so:

```

adjacency_list = {
    'A': [('B', 1), ('C', 3), ('D', 7)],
    'B': [('D', 5)],
    'C': [('D', 12)]
}
graph1 = Graph(adjacency_list)
graph1.a_star_algorithm('A', 'D')

```

And the output would look like:

```

Path found: ['A', 'B', 'D']
['A', 'B', 'D']

```

Thus, the optimal path from **A** to **D**, found using A\*, is **A** -> **B** -> **D**.

```

def a_star_algorithm(self, start_node, stop_node):
    # open_list is a list of nodes which have been visited, but who's neighbors
    # haven't all been inspected, starts off with the start node
    # closed_list is a list of nodes which have been visited
    # and who's neighbors have been inspected
    open_list = set([start_node])
    closed_list = set([])

    # g contains current distances from start_node to all other nodes
    # the default value (if it's not found in the map) is +infinity
    g = {}

    g[start_node] = 0

    # parents contains an adjacency map of all nodes
    parents = {}
    parents[start_node] = start_node

    while len(open_list) > 0:
        n = None

        # find a node with the lowest value of f() - evaluation function
        for v in open_list:
            if n == None or g[v] + self.h(v) < g[n] + self.h(n):
                n = v;

        if n == None:
            print('Path does not exist!')
            return None

```

<https://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/a-star-search-algorithm/>

- Open\_list: 해당 노드를 방문했지만 그 이웃들은 아직 다 방문하지 않았을 때 이 안에 노드를 보관
- Closed\_list: 해당 노드랑 이웃 노드들을 다 방문했을 때 그 노드들을 보관
- Open\_list의 노드들 중에 f값이 더 작은 노드 v가 있으면 n을 v로 교체해준다

```

if n == None:
    print('Path does not exist!')
    return None

# if the current node is the stop_node
# then we begin reconstructin the path from it to the start_node
if n == stop_node:
    reconst_path = []

    while parents[n] != n:
        reconst_path.append(n)
        n = parents[n]

    reconst_path.append(start_node)

    reconst_path.reverse()

    print('Path found: {}'.format(reconst_path))
    return reconst_path

# for all neighbors of the current node do
for (m, weight) in self.get_neighbors(n):
    # if the current node isn't in both open_list and closed_list
    # add it to open_list and note n as it's parent
    if m not in open_list and m not in closed_list:
        open_list.add(m)
        parents[m] = n
        g[m] = g[n] + weight

    # otherwise, check if it's quicker to first visit n, then m
    # and if it is, update parent data and g data
    # and if the node was in the closed_list, move it to open_list

```

<https://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/a-star-search-algorithm/>

- 이렇게 n을 open\_list의 v 값으로 갱신해주다가 n이 목적지 (stop\_node)이면은 이전에 경유한 부모 노드들을 추적하여 reconst\_path에 붙여서 경로를 재구성한다
- N의 이웃한 노드들 m을 가져와서 open\_list에도 closed\_list에도 없으면 open\_list에 추가하고 부모 노드도 기록해둔다

```

# for all neighbors of the current node do
for (m, weight) in self.get_neighbors(n):
    # if the current node isn't in both open_list and closed_list
    # add it to open_list and note n as it's parent
    if m not in open_list and m not in closed_list:
        open_list.add(m)
        parents[m] = n
        g[m] = g[n] + weight

    # otherwise, check if it's quicker to first visit n, then m
    # and if it is, update parent data and g data
    # and if the node was in the closed_list, move it to open_list
    else:
        if g[m] > g[n] + weight:
            g[m] = g[n] + weight
            parents[m] = n

            if m in closed_list:
                closed_list.remove(m)
                open_list.add(m)

# remove n from the open_list, and add it to closed_list
# because all of his neighbors were inspected
open_list.remove(n)
closed_list.add(n)

print('Path does not exist!')
return None

```

ps://stackabuse.com/courses/graphs-in-python-theory-and-implementation/lessons/a-star-search-algorithm/

- 시작 노드에서부터 m까지의 거리( $g[m]$ )가 시작노드부터 n까지의 거리( $g[n]$ )에 m과 n 사이의 거리 (weight)를 더한 값보다 크다면  $g[m]$ 을 더 가까운 거리인  $g[n] + \text{weight}$ 로 갱신
- 이웃한 정점들을 모두 검사했다면 오픈 리스트에서 노드를 제거하고 닫힌 리스트로 넘겨준다